

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



NGUYỄN HỮU QUANG

NHẬN DIỆN LỜI NÓI TỪ THÔNG TIN THỊ GIÁC
TIẾNG VIỆT

ĐỒ ÁN NGÀNH
NGÀNH CÔNG NGHỆ THÔNG TIN

TP. HỒ CHÍ MINH, NĂM 2026

TÓM TẮT ĐỒ ÁN NGÀNH

Nhận dạng lời nói từ thông tin thị giác (*Visual Speech Recognition* - VSR) là bài toán xác định nội dung lời nói thông qua các chuyển động quan sát được trên khuôn mặt, đặc biệt là vùng môi của người nói. So với nhận dạng lời nói dựa trên âm thanh, VSR không phụ thuộc vào tín hiệu âm thanh nhưng vẫn gặp nhiều khó khăn do thông tin thị giác không phản ánh đầy đủ toàn bộ quá trình cấu âm. Điều này dẫn đến sự nhập nhằng giữa các đơn vị lời nói có biểu hiện khẩu hình tương tự nhau.

Đồ án tập trung nghiên cứu bài toán VSR cho tiếng Việt, trong đó chú trọng đến ảnh hưởng của cách biểu diễn đơn vị ngôn ngữ đối với hiệu quả nhận dạng của mô hình. Bên cạnh cách biểu diễn trực tiếp ở cấp độ từ, đề tài xem xét cách biểu diễn dựa trên cấu trúc âm tiết tiếng Việt thông qua việc phân rã đơn vị đầu ra thành các thành phần ngữ âm tương ứng. Các cách biểu diễn này được xây dựng và đánh giá trong cùng điều kiện thực nghiệm nhằm phân tích ảnh hưởng của từng phương pháp đến kết quả nhận dạng.

Hệ thống được xây dựng trên bộ dữ liệu ViCocktail và chỉ sử dụng thông tin thị giác từ video. Trong quá trình xử lý, vùng môi của người nói được xác định và chuẩn hóa trước khi đưa vào mô hình. ShuffleNetV2 được sử dụng để trích xuất đặc trưng thị giác, trong khi Temporal Convolutional Network (TCN) đảm nhiệm việc mô hình hóa sự thay đổi của đặc trưng theo thời gian. Đối với quá trình nhận dạng chuỗi, đề tài sử dụng Connectionist Temporal Classification (CTC) và Multi-label Connectionist Temporal Classification (MCTC) cho các

cách biểu diễn đầu ra tương ứng.

Thông qua các thí nghiệm so sánh, đề án hướng đến việc đánh giá mức độ phù hợp của các cách biểu diễn đơn vị ngôn ngữ đối với bài toán VSR tiếng Việt, đồng thời phân tích khả năng khai thác cấu trúc âm tiết và các đặc trưng ngữ âm của tiếng Việt trong quá trình nhận dạng.

MỤC LỤC

Mục lục	iii
Danh mục hình vẽ	v
Danh mục bảng	vii
1 Giới thiệu	1
1.1 Giới thiệu đề tài	1
1.2 Lý do chọn đề tài	2
1.3 Mục tiêu nghiên cứu	3
1.3.1 Mục tiêu tổng quát	3
1.3.2 Mục tiêu cụ thể	3
1.4 Phạm vi đề tài và đối tượng nghiên cứu	4
1.4.1 Phạm vi đề tài	4
1.4.2 Đối tượng nghiên cứu	4
2 Cơ sở lý thuyết	6
2.1 Tổng quan về Trí tuệ nhân tạo	6
2.1.1 Học sâu	6
2.1.2 Thị giác máy tính	7
2.1.3 Xử lý ngôn ngữ tự nhiên	7

2.2	Những hạn chế thường gặp của hệ thống VSR	8
2.2.1	Những vấn đề phát sinh trong quá trình nhận dạng	8
2.2.2	Những vấn đề liên quan đến dữ liệu	9
2.3	Tổng quan về tiếng Việt	9
2.3.1	Giới thiệu chung	10
2.3.2	Âm vị	10
2.3.3	Âm tiết	10
2.3.4	Các thành phần trong âm tiết	11
2.4	Các công nghệ áp dụng trong đề tài	15
2.4.1	Mạng nơ-ron tích chập	15
2.4.2	Mạng nơ-ron tích chập ba chiều	15
2.4.3	ShuffleNet	16
2.4.4	Temporal Convolutional Network	19
2.4.5	Connectionist Temporal Classification	21
2.4.6	Multi-label Connectionist Temporal Classification	23
2.4.7	Các chỉ số đánh giá	24
3	Hệ thống Nhận diện lời nói từ thông tin thị giác Tiếng Việt	26
3.1	Tổng quan hệ thống	26
3.2	Chuẩn bị dữ liệu	28
3.2.1	Bộ dữ liệu	28
3.2.2	Phân tích dữ liệu	29
3.3	Tiền xử lý dữ liệu	31
3.3.1	Lọc và phân chia dữ liệu	31
3.3.2	VideoTransform	33
3.3.3	Tokenizer	34
3.4	Thiết kế mô hình	38
3.4.1	Kiến trúc tổng thể	38
3.4.2	Visual Frontend	39

3.4.3	Temporal Backend	39
3.4.4	Prediction Head	40
3.4.5	Hàm mất mát	41
3.5	Thiết lập huấn luyện	43
3.6	Kết quả và đánh giá	44
3.6.1	Phân tích quá trình huấn luyện	44
3.6.2	Kết quả trên tập kiểm thử	46
4	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	49
4.1	Kết luận	49
4.2	Hạn chế	50
4.3	Hướng phát triển	51

DANH MỤC HÌNH VẼ

2.1	Cấu trúc âm tiết tiếng Việt	11
2.2	Cơ chế Channel Shuffle với hai lớp Group Convolution xếp chồng.	17
2.3	Các khối cơ bản của ShuffleNet và ShuffleNetV2.	18
2.4	Cấu trúc của khối TCN	20
2.5	Cách hoạt động của Dilated Causal Convolution	21
3.1	Luồng hoạt động của hệ thống VSR.	26
3.2	Luồng hoạt động của quá trình tiền xử lý.	27
3.3	Phân bố dữ liệu theo số lượng khung hình	29
3.4	Phân bố dữ liệu theo số lượng từ	31
3.5	Video sau khi được áp dụng AdaptiveTimeMask	34
3.6	Sự thay đổi của Loss trong quá trình huấn luyện	45
3.7	Sự thay đổi của WER trong quá trình huấn luyện	46

DANH MỤC BẢNG

2.1	Bảng âm vị và ký hiệu quy ước của âm đầu tiếng Việt	12
2.2	Bảng âm vị và ký hiệu quy ước của âm đệm tiếng Việt	12
2.3	Bảng âm vị và ký hiệu quy ước của âm chính tiếng Việt	13
2.4	Bảng âm vị và ký hiệu quy ước của âm cuối tiếng Việt	14
2.5	Bảng thanh điệu tiếng Việt và ký hiệu quy ước	14
3.1	Thống kê dữ liệu trong ViCocktail	28
3.2	Thống kê dữ liệu theo các mức giới hạn số lượng khung hình . .	32
3.3	Ảnh hưởng của các mức giới hạn số lượng khung hình đến dữ liệu phiên âm	32
3.4	Phân chia dữ liệu sau quá trình lọc	33
3.5	Kết quả đánh giá của hai mô hình trên tập kiểm thử	47

Chương 1

GIỚI THIỆU

1.1 Giới thiệu đề tài

Nhận dạng lời nói tự động (**Automatic Speech Recognition - ASR**) là một bài toán quan trọng với mục tiêu chuyển đổi nội dung lời nói của con người thành văn bản tương ứng. Đây là nền tảng cho nhiều ứng dụng quen thuộc như: trợ lý ảo, nhập liệu tự động,... Hiện nay, trong điều kiện lý tưởng, các hệ thống ASR hiện đại đã có thể đưa ra những dự đoán vô cùng chính xác, trong một số trường hợp có thể tiệm cận với khả năng của con người. Tuy nhiên, phần lớn các hệ thống này vẫn phụ thuộc chủ yếu vào tín hiệu âm thanh, vốn dễ bị ảnh hưởng bởi nhiều yếu tố: môi trường, thiết bị thu âm,... Những yếu tố này thường gây suy giảm chất lượng của tín hiệu, kéo theo khả năng dự đoán của hệ thống bị giảm sút.

Trên thực tế, lời nói không chỉ được truyền tải qua âm thanh mà còn thể hiện qua các chuyển động quan sát được trên khuôn mặt người nói. Đây là cơ sở của bài toán **nhận dạng lời nói từ thông tin thị giác (Visual Speech Recognition - VSR)**. Tuy VSR giúp giảm sự phụ thuộc vào tín hiệu âm thanh và hạn chế những vấn đề trên, việc chỉ khai thác thông tin thị giác cũng đặt ra nhiều thách thức riêng. Một trong những thách thức đó là việc khẩu hình không phản ánh đầy đủ quá trình cấu âm, dẫn đến thông tin thị giác thường không

đủ để phân biệt chính xác các âm vị khác nhau. Vì vậy, nhiều âm vị có thể tạo ra những biểu hiện khẩu hình tương tự nhau, dẫn đến sự nhập nhằng trong quá trình nhận dạng.

Nhận biết được vấn đề này, đề tài tập trung nghiên cứu một phương pháp nhằm cải thiện độ chính xác của mô hình VSR, với đối tượng nghiên cứu là tiếng Việt. Đây là một ngôn ngữ có những đặc trưng riêng về mặt ngữ âm, một trong những yếu tố làm tăng độ khó của bài toán, nhưng đồng thời cũng mở ra nhiều hướng tiếp cận mới.

1.2 Lý do chọn đề tài

Trong những năm gần đây, nhận dạng lời nói từ thông tin thị giác ngày càng nhận được nhiều sự quan tâm. Một phần là nhờ sự phát triển của các phương pháp *Học Sâu (Deep Learning - DL)* thuộc lĩnh vực *Trí tuệ nhân tạo (Artificial Intelligence - AI)* đã giúp các máy tính có thể hiểu sâu hơn về bài toán. Tuy nhiên, phần lớn các nghiên cứu vẫn tập trung vào một số ngôn ngữ có nhiều tài nguyên dữ liệu, đặc biệt là tiếng Anh [1]. Điều này dẫn đến sự chênh lệch đáng kể về mức độ phát triển giữa các ngôn ngữ. Đối với tiếng Việt, số lượng nghiên cứu về VSR trước đây còn tương đối hạn chế, một phần do thiếu các bộ dữ liệu có quy mô đủ lớn để phục vụ việc huấn luyện và đánh giá mô hình. Gần đây, sự xuất hiện của các bộ dữ liệu tiếng Việt dành riêng cho bài toán [2, 3] đã tạo điều kiện thuận lợi cho việc mở rộng nghiên cứu.

Bên cạnh sự phát triển về dữ liệu, tiếng Việt cũng có những đặc trưng ngữ âm riêng biệt. Đáng chú ý, tiếng Việt sử dụng thanh điệu như một thành phần quan trọng trong việc cấu tạo nên âm tiết[4]. Việc thay đổi thanh điệu có thể dẫn đến thay đổi nghĩa của từ, ví dụ như: ma và mà, cạnh và cánh,.. Ngoài nét đặc trưng riêng của thanh điệu, âm tiết tiếng Việt có cấu trúc tương đối chặt chẽ, với nhiều thành phần đảm nhiệm chức năng khác nhau.

Từ những cơ sở trên, đề tài lựa chọn hướng nghiên cứu cách biểu diễn

đơn vị ngôn ngữ cho bài toán VSR tiếng Việt, trong đó xem xét khả năng khai thác cấu trúc âm tiết và các đặc trưng ngữ âm của tiếng Việt trong quá trình nhận dạng. Trên cơ sở đó, đề tài tiến hành xây dựng và so sánh các cách biểu diễn khác nhau nhằm đánh giá ảnh hưởng của chúng đến hiệu quả nhận dạng.

1.3 Mục tiêu nghiên cứu

1.3.1 Mục tiêu tổng quát

Nghiên cứu và đánh giá một hướng tiếp cận phù hợp cho bài toán VSR tiếng Việt, trong đó tập trung khai thác các đặc trưng ngôn ngữ và cấu trúc âm tiết của tiếng Việt nhằm cải thiện khả năng biểu diễn và nhận dạng của mô hình.

1.3.2 Mục tiêu cụ thể

Trong phạm vi nghiên cứu của đề tài, các mục tiêu cụ thể được xác định như sau:

- **Xây dựng mô hình nhận dạng VSR cho tiếng Việt:** Triển khai mô hình VSR trên dữ liệu tiếng Việt, làm cơ sở cho quá trình nghiên cứu và đánh giá các phương pháp biểu diễn khác nhau.
- **Nghiên cứu cách biểu diễn đơn vị ngôn ngữ phù hợp:** Khảo sát và xây dựng cách biểu diễn đơn vị đầu ra dựa trên các đặc trưng và cấu trúc âm tiết của tiếng Việt, nhằm khai thác tốt hơn các thông tin ngôn ngữ trong quá trình nhận dạng.
- **Đánh giá ảnh hưởng của cách biểu diễn:** Thực hiện các thí nghiệm nhằm so sánh các cách biểu diễn đơn vị ngôn ngữ trong cùng điều kiện huấn luyện và đánh giá, từ đó xác định ảnh hưởng của chúng đến hiệu quả nhận dạng.

- **Phân tích kết quả và hạn chế:** Phân tích kết quả thực nghiệm nhằm làm rõ ưu điểm và hạn chế của từng hướng tiếp cận, đồng thời đề xuất các hướng cải thiện và phát triển tiếp theo.

1.4 Phạm vi đề tài và đối tượng nghiên cứu

1.4.1 Phạm vi đề tài

Đề tài tập trung nghiên cứu bài toán nhận dạng lời nói từ thông tin thị giác cho tiếng Việt, trong đó xem xét ảnh hưởng của cách biểu diễn đơn vị ngôn ngữ đến kết quả nhận dạng. Hai cách biểu diễn được khảo sát gồm biểu diễn ở cấp độ từ và biểu diễn dựa trên cấu trúc âm tiết tiếng Việt với ba thành phần âm đầu, vần và thanh điệu.

Các thí nghiệm được thực hiện trên bộ dữ liệu ViCocktail [2], sử dụng chuỗi khung hình vùng môi và nội dung phiên âm tương ứng. Tín hiệu âm thanh không được sử dụng trong quá trình huấn luyện và đánh giá.

Các phương pháp được xây dựng và đánh giá trong cùng điều kiện thực nghiệm nhằm đảm bảo tính nhất quán khi so sánh. Đề tài không tập trung vào việc triển khai hệ thống trong môi trường thực tế và không nghiên cứu bài toán *Audio-Visual Speech Recognition* (AVSR).

1.4.2 Đối tượng nghiên cứu

Đối tượng nghiên cứu chính của đề tài bao gồm:

- **Dữ liệu VSR tiếng Việt:** chuỗi khung hình vùng môi và nội dung phiên âm tương ứng trong bộ dữ liệu ViCocktail.
- **Mô hình VSR:** quá trình trích xuất đặc trưng thị giác, mô hình hóa thông tin theo thời gian và tạo chuỗi dự đoán.
- **Cách biểu diễn đơn vị ngôn ngữ:** biểu diễn ở cấp độ từ và biểu diễn

dựa trên các thành phần âm đầu, vần và thanh điệu.

- **Kết quả nhận dạng:** sự khác biệt về hiệu quả nhận dạng giữa các cách biểu diễn trong cùng điều kiện thực nghiệm.

CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về Trí tuệ nhân tạo

Trí tuệ nhân tạo (Artificial Intelligence - AI) là lĩnh vực nghiên cứu hướng đến việc xây dựng các hệ thống có khả năng thực hiện những tác vụ thường đòi hỏi khả năng nhận thức, suy luận hoặc ra quyết định của con người. Phạm vi của AI bao gồm nhiều hướng nghiên cứu khác nhau và có thể được ứng dụng trên nhiều dạng dữ liệu như hình ảnh, video, âm thanh và văn bản.

Trong phạm vi đề tài, các nội dung có liên quan trực tiếp đến bài toán nhận dạng lời nói từ thông tin thị giác gồm *Học sâu*, *Thị giác máy tính* và *Xử lý ngôn ngữ tự nhiên*. Trong đó, học sâu cung cấp nền tảng cho việc xây dựng mô hình, còn hai lĩnh vực còn lại liên quan đến quá trình xử lý thông tin thị giác và biểu diễn thông tin ngôn ngữ.

2.1.1 Học sâu

Học sâu (Deep Learning - DL) là một nhánh của **học máy (Machine Learning - ML)**, trong đó các mạng nơ-ron sâu được sử dụng để học và biến đổi dữ liệu qua nhiều tầng liên tiếp. Qua quá trình này, mô hình có thể hình thành các biểu diễn phù hợp với nhiệm vụ cần giải quyết.

Một đặc điểm quan trọng của học sâu là khả năng tự động học các đặc trưng trực tiếp từ dữ liệu thông qua quá trình huấn luyện. Các tầng đầu thường học những đặc trưng ở mức đơn giản, trong khi các tầng sâu hơn tiếp tục kết hợp và biến đổi chúng để hình thành những biểu diễn phức tạp hơn. Nhờ khả năng này, học sâu được ứng dụng rộng rãi trong xử lý hình ảnh, video, âm thanh và ngôn ngữ, đồng thời là nền tảng cho các kiến trúc được sử dụng trong đề tài.

2.1.2 Thị giác máy tính

Đối với dữ liệu hình ảnh và video, **Thị giác máy tính (Computer Vision - CV)** tập trung vào việc giúp máy tính phân tích và khai thác các thông tin xuất hiện trong dữ liệu thị giác. Các phương pháp trong lĩnh vực này cho phép hệ thống thực hiện nhiều nhiệm vụ khác nhau, tiêu biểu như phân loại hình ảnh, phát hiện và theo dõi đối tượng, phân đoạn ảnh và phân tích nội dung video.

2.1.3 Xử lý ngôn ngữ tự nhiên

Ở phía dữ liệu ngôn ngữ, **Xử lý ngôn ngữ tự nhiên (Natural Language Processing - NLP)** hướng đến việc xây dựng các phương pháp giúp máy tính xử lý, biểu diễn và khai thác thông tin từ ngôn ngữ của con người. Tùy theo bài toán, dữ liệu có thể được biểu diễn ở nhiều cấp độ khác nhau như ký tự, từ, câu hoặc văn bản.

Từ các dạng biểu diễn này, NLP được ứng dụng trong nhiều bài toán như phân loại văn bản, dịch máy, tóm tắt văn bản, trả lời câu hỏi và sinh văn bản.

2.2 Những hạn chế thường gặp của hệ thống VSR

Như đã trình bày ở Chương 1, các hệ thống VSR vẫn tồn tại nhiều hạn chế, xuất phát từ cả bản chất của tín hiệu thị giác lẫn đặc điểm của dữ liệu được sử dụng. Dựa trên cách phân loại của Sheng và các cộng sự, mục này tập trung trình bày hai nhóm vấn đề có liên quan trực tiếp đến phạm vi nghiên cứu của đề tài, bao gồm những khó khăn phát sinh trong quá trình nhận dạng và những hạn chế liên quan đến dữ liệu [1].

2.2.1 Những vấn đề phát sinh trong quá trình nhận dạng

Một trong những thách thức trong quá trình nhận dạng là việc thông tin thị giác không phản ánh đầy đủ toàn bộ quá trình cấu âm. Các đặc điểm như sự rung của dây thanh quản hay một số chuyển động của lưỡi trong quá trình phát âm thường không được quan sát đầy đủ khi chỉ thông qua mỗi khẩu hình [5]. Do đó, nhiều âm vị khác nhau có thể tạo ra các biểu hiện thị giác tương tự nhau, còn được gọi là các *viseme* [6]. Sự tương đồng này làm giảm khả năng phân biệt của mô hình nhận dạng đối với các đơn vị lời nói, đặc biệt khi phạm vi từ vựng ngày càng được mở rộng. Hiện tượng này được gọi là **sự tương đồng giữa các lớp (Inter-class Similarities)** [1].

Trái ngược với nhóm trên, cùng một nội dung lời nói cũng có thể tạo ra những đặc trưng thị giác khác nhau. Với cùng một người nói, một từ có thể được thể hiện bằng những chuyển động môi khác nhau tùy theo ngữ cảnh, tốc độ nói hay trạng thái cảm xúc. Ngoài ra, điều kiện ghi hình như góc quay, ánh sáng, chất lượng hình ảnh cũng ảnh hưởng trực tiếp đến đặc trưng thu được. Nhóm khó khăn này thường được gọi là **sự biến thiên trong cùng Một Lớp (Intra-class Variations)** [1], thường khiến mô hình nhận dạng khó học được những mẫu đặc trưng ổn định.

2.2.2 Những vấn đề liên quan đến dữ liệu

Bên cạnh những khó khăn phát sinh trong quá trình nhận dạng, dữ liệu phục vụ cho bài toán VSR cũng tồn tại một số hạn chế đáng chú ý. Trước hết là về quy mô, việc xây dựng dữ liệu đòi hỏi thu thập và xử lý đồng thời hình ảnh của người nói cùng với nội dung lời nói tương ứng, kéo theo chi phí đáng kể về thời gian và công sức, đồng thời còn vướng các vấn đề liên quan đến quyền riêng tư của người tham gia. Ngoài ra, việc xây dựng chú thích cho một lượng lớn dữ liệu cũng tốn nhiều công sức, khiến dữ liệu thu thập được không phải lúc nào cũng đi kèm đầy đủ thông tin chú thích cần thiết cho quá trình huấn luyện [1].

Một hạn chế nữa nằm ở mức độ đa dạng giữa các miền dữ liệu. Khác với dữ liệu văn bản, lời nói và hình ảnh khuôn mặt có thể thay đổi đáng kể theo ngôn ngữ, độ tuổi, giới tính và đặc điểm riêng của từng người nói [1]. Trong thực tế, các bộ dữ liệu VSR thường chỉ bao phủ được một phần trong số những biến đổi này, khiến phân bố dữ liệu huấn luyện chưa đại diện đầy đủ cho các trường hợp xuất hiện trong môi trường thực tế. Khi gặp những mẫu có đặc điểm khác với dữ liệu mà mô hình được học trong quá trình huấn luyện, khả năng nhận dạng của mô hình có thể bị suy giảm đáng kể.

2.3 Tổng quan về tiếng Việt

Bên cạnh những hạn chế xuất phát từ tín hiệu thị giác và dữ liệu, đặc điểm của ngôn ngữ được nhận dạng cũng cần được xem xét khi xây dựng hệ thống. Vì vậy, phần này trình bày một số đặc điểm cơ bản của tiếng Việt, tập trung vào hệ thống ngữ âm và cấu trúc âm tiết, làm cơ sở cho cách tổ chức và phân rã nhãn được sử dụng trong đề tài. Các nội dung về ngữ âm và cấu trúc âm tiết được tham chiếu theo hệ thống phân tích của Đoàn Thiện Thuật trong *Ngữ âm tiếng Việt* [4].

2.3.1 Giới thiệu chung

Tiếng Việt là ngôn ngữ chính thức của Việt Nam, giữ vai trò chủ đạo trong giao tiếp, giáo dục, hành chính và đời sống xã hội. Xét về nguồn gốc, tiếng Việt thuộc nhóm Vietic của ngữ hệ Nam Á (Austroasiatic) [7]. Đây là một ngữ hệ gồm nhiều ngôn ngữ được phân bố chủ yếu tại khu vực Đông Nam Á và một số vùng lân cận.

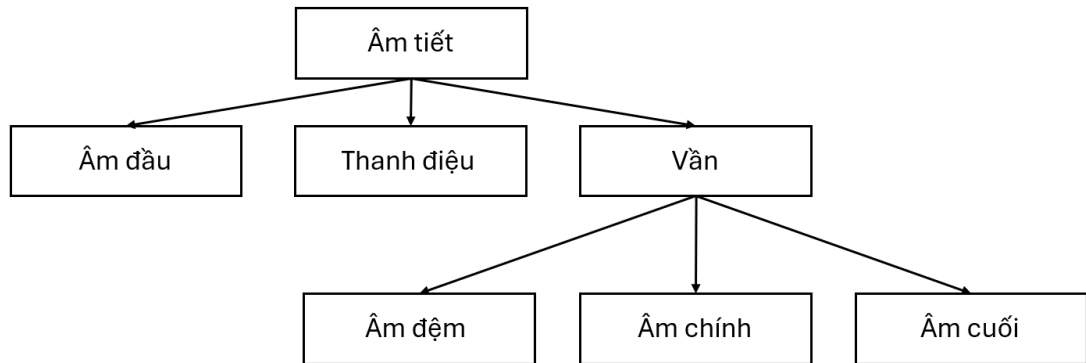
Trong quá trình hình thành và phát triển, tiếng Việt chịu ảnh hưởng từ quá trình tiếp xúc với nhiều ngôn ngữ khác, đặc biệt là tiếng Trung. Những tác động này, cùng với quá trình phát triển nội tại của ngôn ngữ, đã góp phần hình thành nên những đặc trưng riêng của tiếng Việt, trong đó có các đặc điểm về ngữ âm và cấu trúc âm tiết.

2.3.2 Âm vị

Âm vị là đơn vị nhỏ nhất trong hệ thống ngữ âm của một ngôn ngữ. Âm vị có chức năng khu biệt nghĩa [4], tức là sự khác biệt giữa các âm vị có thể tạo nên sự khác biệt về nghĩa giữa các từ hoặc âm tiết. Trong tiếng Việt, các âm vị được tổ chức và kết hợp theo những quy tắc nhất định, qua đó góp phần hình thành nên cấu trúc chặt chẽ của âm tiết.

2.3.3 Âm tiết

Âm tiết là đơn vị phát âm nhỏ nhất trong lời nói. Trong tiếng Việt, âm tiết có cấu trúc tương đối chặt chẽ, trong đó mỗi thành phần đảm nhận một vị trí và chức năng nhất định. Theo hệ thống phân tích của Đoàn Thiện Thuật, ở cấp độ tổng quát, một âm tiết tiếng Việt có thể được phân tích thành ba thành phần chính gồm **âm đầu**, **vần** và **thanh điệu**. Trong đó, phần vần có thể tiếp tục được phân chia thành **âm đệm**, **âm chính** và **âm cuối** [4]. Cấu trúc và mối quan hệ giữa các thành phần của âm tiết tiếng Việt được minh họa trong Hình 2.1.



Hình 2.1: Cấu trúc âm tiết tiếng Việt

2.3.4 Các thành phần trong âm tiết

Phần này trình bày các thành phần trong cấu trúc âm tiết tiếng Việt, đồng thời cung cấp các ký hiệu quy ước được sử dụng trong đề tài.

Âm đầu

Âm đầu là thành phần xuất hiện ở vị trí đầu của âm tiết và thường được đảm nhiệm bởi một phụ âm. Theo Đoàn Thiện Thuật, tiếng Việt có 22 âm vị, tương ứng với 26 chữ viết[4]. Các âm đầu và ký hiệu quy ước được sử dụng trong đề tài được trình bày trong Bảng 2.1.

IPA	Con chữ	Quy ước	IPA	Con chữ	Quy ước
/b/	b	/b/	/t̚/	tr	/tr/
/m/	m	/m/	/s̺/	s	/s/
/f/	ph	/f/	/z̺/	r	/r/
/v/	v	/v/	/c/	ch	/c/
/tʰ/	th	/th/	/ɲ/	nh	/nh/
/t/	t	/t/	/k/	c, k, q	/k/
/d/	đ	/d/	/ŋ/	ng, ngh	/ng/
/n/	n	/n/	/χ/	kh	/xh/
/s/	x	/sx/	/ɣ/	g, gh	/yg/
/z/	d, gi	/z/	/ʔ/	-	/null/
/l/	l	/l/	/h/	h	/h/

Bảng 2.1: Bảng âm vị và ký hiệu quy ước của âm đầu tiếng Việt

Âm đệm

Âm đệm là thành phần nằm giữa âm đầu và âm chính, thuộc phần vần của âm tiết. Trong tiếng Việt, âm đệm được biểu hiện bởi âm vị /w/, thường được thể hiện trong chữ viết bằng *o* hoặc *u* [4]. Một số âm tiết không có âm đệm, khi đó vị trí này được xem là khuyết. Các âm đệm và ký hiệu quy ước được sử dụng trong đề tài được trình bày trong Bảng 2.2.

IPA	Con chữ	Quy ước
/w/	o, u	/-u-/
∅	-	/zero/

Bảng 2.2: Bảng âm vị và ký hiệu quy ước của âm đệm tiếng Việt

Âm chính

Âm chính là thành phần hạt nhân của phần vần trong âm tiết tiếng Việt. Thành phần này được đảm nhiệm bởi nguyên âm và giữ vai trò chính trong việc tạo nên âm sắc của âm tiết [4]. Âm chính có thể được biểu hiện bằng nguyên âm đơn hoặc nguyên âm đôi. Các âm chính và ký hiệu quy ước được sử dụng trong đề tài được trình bày trong Bảng 2.3.

IPA	Con chữ	Quy ước	IPA	Con chữ	Quy ước
/i/	i, y	/i/	/ǣ/	â	/aa/
/e/	ê	/ee/	/ǣ̃/	a, ă	/aw/
/ɛ/	e	/e/	/uɤ/	ơ, ư	/wow/
/ɛ̃/	a	/ew/	/u/	u	/u/
/ie/	iê, ia, yê, ya	/ie/	/o/	ô, ơ	/oo/
/u/	ư	/w/	/ɔ/	oo, o	/o/
/ɤ/	ơ	/ow/	/ɔ̃/	o	/o_short/
/a/	a	/a/	/uo/	uô, ua	/uo/

Bảng 2.3: Bảng âm vị và ký hiệu quy ước của âm chính tiếng Việt

Âm cuối

Âm cuối là thành phần nằm ở vị trí cuối của phần vần, có chức năng kết thúc âm tiết. Theo Đoàn Thiện Thuật, âm cuối tiếng Việt có thể được đảm nhiệm bởi phụ âm, bán nguyên âm hoặc có thể khuyết [4]. Các âm cuối và ký hiệu quy ước được sử dụng trong đề tài được trình bày trong Bảng 2.4.

IPA	Con chữ	Quy ước	IPA	Con chữ	Quy ước
/-p/	p	/-p/	/-ŋ/	ng, nh	/-ng/
/-t/	t	/-t/	/-u/	u, o	/-u/
/-k/	c, ch	/-k/	/-i/	i, y	/-i/
/-m/	m	/-m/	∅	-	/zero/
/-n/	n	/-n/			

Bảng 2.4: Bảng âm vị và ký hiệu quy ước của âm cuối tiếng Việt

Thanh điệu

Thanh điệu là thành phần có tính chất *siêu đoạn tính*, tức là không gắn với một âm vị riêng lẻ mà được thể hiện trên toàn bộ âm tiết. Thanh điệu có chức năng khu biệt nghĩa trong tiếng Việt [4], ví dụ như với cùng cấu trúc âm tiết “ma”, sự thay đổi thanh điệu có thể tạo nên các từ khác nhau như “ma”, “mà”, “má”, “mả”, “mã” và “mạ”. Tiếng Việt có sáu thanh điệu gồm ngang, huyền, sắc, hỏi, ngã và nặng. Trong chữ viết, năm thanh được biểu diễn bằng dấu thanh, riêng thanh ngang không có dấu. Các thanh điệu và ký hiệu quy ước được sử dụng trong đề tài được trình bày trong Bảng 2.5.

Thanh điệu	Quy ước	Thanh điệu	Quy ước
Ngang	/ngang/	Hỏi	/hoi/
Huyền	/huyen/	Ngã	/nga/
Sắc	/sac/	Nặng	/nang/

Bảng 2.5: Bảng thanh điệu tiếng Việt và ký hiệu quy ước

2.4 Các công nghệ áp dụng trong đề tài

2.4.1 Mạng nơ-ron tích chập

Mạng nơ-ron tích chập (Convolutional Neural Network - CNN) là một kiến trúc mạng nơ-ron sâu được phát triển nhằm xử lý hiệu quả các dữ liệu có cấu trúc dạng lưới, đặc biệt là hình ảnh. Thành phần cốt lõi của CNN là lớp tích chập (*Convolution Layer*). Tại đây, các bộ lọc có kích thước nhỏ, ví dụ như 3×3 hoặc 5×5 , được lần lượt áp dụng lên dữ liệu đầu vào nhằm trích xuất những đặc trưng khác nhau.

Kết quả của quá trình này tạo thành các bản đồ đặc trưng (*feature maps*), trong đó mỗi bộ lọc có thể phản ánh một dạng đặc trưng nhất định của dữ liệu. Do cùng một bộ lọc được sử dụng tại nhiều vị trí khác nhau, CNN có thể khai thác các đặc trưng cục bộ trong khi vẫn hạn chế số lượng tham số cần học. Phép toán của lớp tích chập có thể được biểu diễn như sau [8]:

$$Z_{i,j}^{(k)} = \sum_{m=1}^M \sum_{n=1}^N X_{(i+m-1),(j+n-1)} W_{m,n}^{(k)} + b^{(k)}, \quad (2.1)$$

Trong đó X là dữ liệu đầu vào, $W^{(k)}$ là bộ lọc thứ k có kích thước $M \times N$, $b^{(k)}$ là hệ số bias và $Z_{i,j}^{(k)}$ là giá trị đầu ra tại vị trí (i, j) trên bản đồ đặc trưng thứ k .

2.4.2 Mạng nơ-ron tích chập ba chiều

Mạng nơ-ron tích chập ba chiều (Three-Dimensional Convolutional Neural Network - 3D CNN) là một dạng mở rộng của CNN hai chiều, trong đó phép tích chập được thực hiện trên cả hai chiều không gian và chiều thời gian [9].

Về mặt toán học, phép tích chập ba chiều có thể được biểu diễn như sau:

$$Z_{t,i,j}^{(k)} = \sum_{p=1}^P \sum_{m=1}^M \sum_{n=1}^N X_{(t+p-1),(i+m-1),(j+n-1)} W_{p,m,n}^{(k)} + b^{(k)}, \quad (2.2)$$

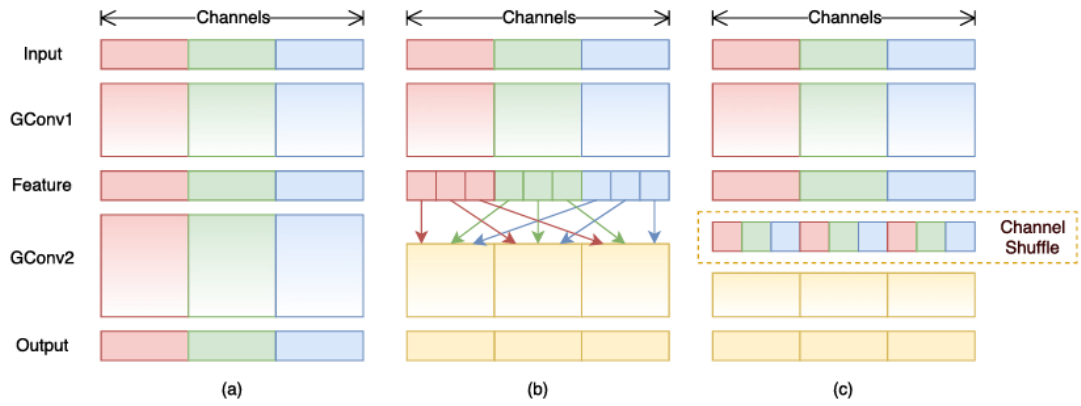
Trong đó X là dữ liệu đầu vào, $W^{(k)}$ là bộ lọc ba chiều thứ k có kích thước $P \times M \times N$, với P biểu diễn kích thước của bộ lọc theo chiều thời gian, M và N lần lượt là kích thước theo hai chiều không gian. $b^{(k)}$ là hệ số bias, còn $Z_{t,i,j}^{(k)}$ là giá trị đầu ra tại thời điểm t và vị trí không gian (i, j) trên bản đồ đặc trưng thứ k .

2.4.3 ShuffleNet

ShuffleNet là kiến trúc mạng nơ-ron tích chập được thiết kế nhằm giảm chi phí tính toán, hướng đến các hệ thống có tài nguyên hạn chế. Một trong những thành phần có chi phí đáng kể trong CNN là phép tích chập 1×1 , đặc biệt khi số lượng kênh lớn. Để giảm chi phí này, ShuffleNet sử dụng *pointwise group convolution* kết hợp với cơ chế *channel shuffle* [10].

Pointwise Group Convolution: Thay vì sử dụng toàn bộ các kênh đầu vào trong mỗi phép tích chập 1×1 , *pointwise group convolution* chia các kênh thành nhiều nhóm và thực hiện phép tích chập độc lập trên từng nhóm. Cách tổ chức này giúp giảm số lượng phép tính và tham số của mạng [10].

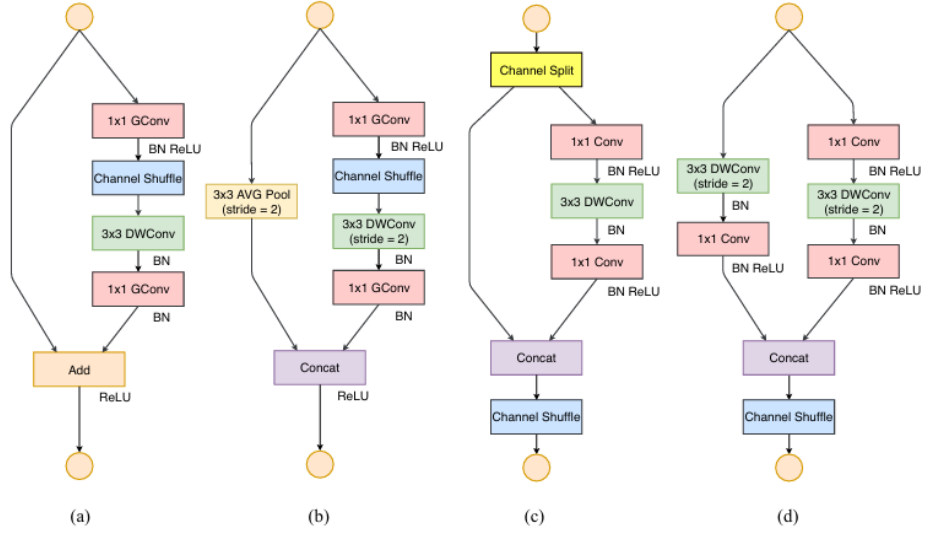
Tuy nhiên, khi nhiều lớp *group convolution* được xếp chồng, thông tin giữa các nhóm có thể bị hạn chế. ShuffleNet khắc phục vấn đề này bằng cơ chế *channel shuffle*, trong đó các kênh được sắp xếp lại trước khi đưa vào lớp tiếp theo, qua đó cho phép thông tin được trao đổi giữa các nhóm khác nhau [10].



Hình 2.2: Cơ chế Channel Shuffle với hai lớp Group Convolution xếp chồng.

ShuffleNet Unit: Khối ShuffleNet kết hợp phép tích chập 1×1 theo nhóm, *channel shuffle* và *depthwise convolution* 3×3 . Đối với khối thông thường, kết nối *residual* được sử dụng để truyền đặc trưng từ đầu vào đến đầu ra. Khi thực hiện *downsampling*, kiến trúc sử dụng hai nhánh và nối kết quả theo chiều kênh bằng phép *concatenation* [10].

Mặc dù ShuffleNet giúp giảm đáng kể số lượng phép tính, số FLOPs không phản ánh hoàn toàn tốc độ thực thi thực tế. Các yếu tố như chi phí truy cập bộ nhớ và mức độ phân mảnh của kiến trúc cũng ảnh hưởng đến hiệu quả tính toán. Trên cơ sở đó, ShuffleNetV2 được đề xuất với cách tổ chức các khối xử lý đơn giản và hiệu quả hơn [11].



Hình 2.3: Các khối cơ bản của ShuffleNet và ShuffleNetV2.

ShuffleNetV2: ShuffleNetV2 hạn chế việc sử dụng *group convolution* trong các phép tích chập 1×1 . Thay vào đó, kiến trúc sử dụng *channel split*, các phép tích chập 1×1 thông thường, *depthwise convolution*, *concatenation* và *channel shuffle* [11]. Cấu trúc các khối của ShuffleNet và ShuffleNetV2 được minh họa trong Hình 2.3.

Channel Split: Trong khối ShuffleNetV2 không thực hiện *downsampling*, đặc trưng đầu vào được chia thành hai nhánh có số lượng kênh bằng nhau, như minh họa trong Hình 2.3(c). Một nhánh được truyền trực tiếp đến đầu ra, trong khi nhánh còn lại lần lượt đi qua phép tích chập 1×1 , *depthwise convolution* 3×3 và một phép tích chập 1×1 khác [11].

Sau đó, hai nhánh được nối lại bằng phép *concatenation* và *channel shuffle* được áp dụng để trao đổi thông tin giữa các kênh trước khi chuyển sang khối tiếp theo.

Downsampling Unit: Khi cần giảm kích thước không gian, ShuffleNetV2 sử dụng cấu trúc hai nhánh như Hình 2.3(d). Trong trường hợp này, *channel*

split không được sử dụng và cả hai nhánh đều nhận toàn bộ đặc trưng đầu vào. Các nhánh sử dụng *depthwise convolution* với *stride* bằng 2 để giảm kích thước không gian, sau đó đầu ra được nối lại theo chiều kênh và đưa qua *channel shuffle* [11].

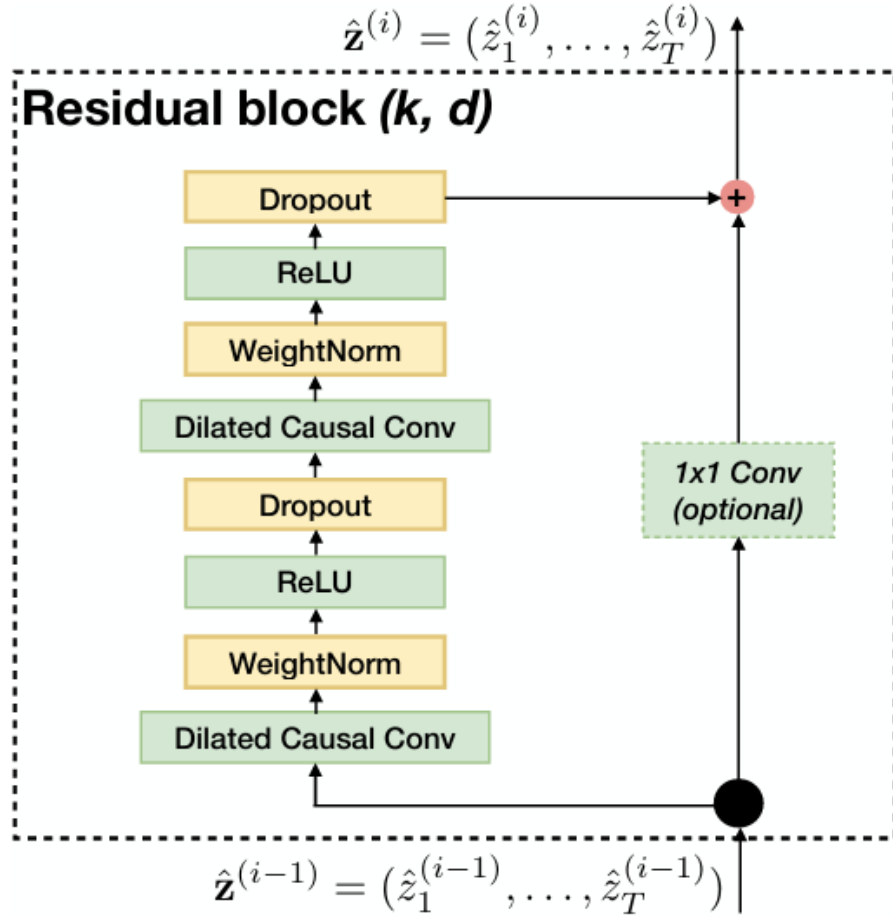
Nhìn chung, ShuffleNetV2 kế thừa cơ chế *channel shuffle* của ShuffleNet nhưng thay đổi cách tổ chức các khối thông qua *channel split* và giảm việc sử dụng *group convolution*. Những thay đổi này giúp cải thiện hiệu quả thực thi của mạng trên phần cứng thực tế [11].

2.4.4 Temporal Convolutional Network

Temporal Convolutional Network (TCN) là một kiến trúc mạng nơ-ron tích chập được thiết kế cho bài toán xử lý dữ liệu dạng chuỗi. Khác với các mạng nơ-ron hồi quy (*Recurrent Neural Network* - RNN) xử lý các phần tử theo trình tự, TCN sử dụng phép tích chập một chiều trên trục thời gian để khai thác mối quan hệ giữa các phần tử trong chuỗi. Cấu trúc của khối TCN được minh họa trong Hình 2.4, trích từ [12].

Causal Convolution: *Causal Convolution* là phép tích chập trong đó đầu ra tại thời điểm t chỉ phụ thuộc vào phần tử hiện tại và các phần tử xuất hiện trước đó. Thông tin từ các thời điểm trong tương lai không được sử dụng, qua đó đảm bảo tính nhân quả của chuỗi [12].

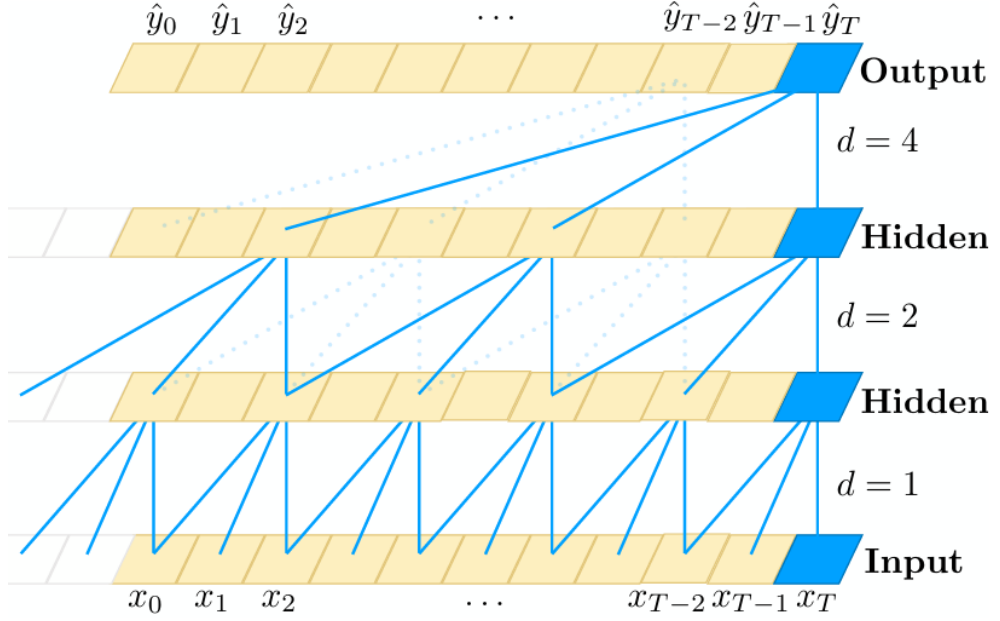
Dilated Convolution: *Dilated Convolution* là phép tích chập trong đó các phần tử của kernel được áp dụng lên đầu vào với một khoảng cách nhất định giữa các vị trí. Khoảng cách này được xác định bởi hệ số giãn d (*dilation factor*), cho phép mở rộng vùng tiếp nhận (*receptive field*) mà không cần tăng kích thước kernel [13]. Khi $d = 1$, *Dilated Convolution* tương đương với phép tích chập thông thường.



Hình 2.4: Cấu trúc của khối TCN

Dilated Causal Convolution: Trong TCN, *Causal Convolution* và *Dilated Convolution* được kết hợp thành *Dilated Causal Convolution*. Sự kết hợp này vừa đảm bảo đầu ra tại mỗi thời điểm không phụ thuộc vào thông tin trong tương lai, vừa cho phép vùng tiếp nhận được mở rộng theo chiều thời gian. Nhờ đó, mô hình có thể khai thác thông tin từ các thời điểm cách xa nhau mà không cần sử dụng kernel có kích thước lớn [12]. Cách hoạt động của *Dilated Causal Convolution* được minh họa trong Hình 2.5, trích từ [12].

Trong TCN, hệ số giãn thường được tăng dần theo độ sâu của mạng, chẳng hạn $d = 1, 2, 4, \dots$. Khi nhiều lớp tích chập được xếp chồng, vùng tiếp nhận của mô hình được mở rộng nhanh chóng, từ đó cho phép khai thác các



Hình 2.5: Cách hoạt động của Dilated Causal Convolution

mối quan hệ trên một khoảng thời gian dài hơn trong chuỗi [12].

Residual Block: Khi nhiều lớp tích chập được xếp chồng, quá trình huấn luyện mạng có thể trở nên khó khăn hơn. Để hỗ trợ việc truyền thông tin và gradient qua các lớp, TCN sử dụng kết nối *residual*. Cơ chế này được đề xuất bởi He và các cộng sự, trong đó đầu vào của một khối được kết nối trực tiếp với đầu ra sau phép biến đổi [14]. Cơ chế *residual* có thể được biểu diễn như sau:

$$y = \mathcal{F}(x) + x, \quad (2.3)$$

trong đó x là đầu vào của khối, $\mathcal{F}(x)$ là phép biến đổi được thực hiện bởi các lớp bên trong khối và y là đầu ra sau khi thực hiện kết nối residual.

2.4.5 Connectionist Temporal Classification

Connectionist Temporal Classification (CTC) là một phương pháp được sử dụng trong các bài toán nhận dạng chuỗi khi mối tương ứng giữa chuỗi đặc trưng đầu vào và chuỗi nhãn mục tiêu chưa được xác định trước [15, 16].

Thay vì yêu cầu thông tin căn chỉnh (*alignment*) tại từng thời điểm, CTC xem xét các khả năng căn chỉnh khác nhau trong quá trình huấn luyện.

Giả sử chuỗi đầu vào có độ dài T được biểu diễn bởi

$$X = (x_1, x_2, \dots, x_T), \quad (2.4)$$

và chuỗi nhãn mục tiêu có độ dài U là

$$z = (z_1, z_2, \dots, z_U), \quad (2.5)$$

với $U \leq T$. Nếu có K nhãn cần dự đoán, CTC bổ sung thêm một nhãn đặc biệt gọi là *blank*, ký hiệu là $\emptyset$. Vì vậy, tại mỗi thời điểm t , mô hình tạo ra một phân phối xác suất trên $K + 1$ nhãn [16].

Thay vì xác định trực tiếp vị trí tương ứng của từng nhãn mục tiêu, CTC xét các chuỗi trung gian có cùng độ dài T với chuỗi đầu vào, được gọi là *path*. Một path được biểu diễn bởi

$$p = (p_1, p_2, \dots, p_T), \quad (2.6)$$

trong đó mỗi p_t có thể là một nhãn thông thường hoặc blank. Nếu $y_t^{p_t}$ là xác suất dự đoán nhãn p_t tại thời điểm t , xác suất của một path được xác định bởi

$$P(p|X) = \prod_{t=1}^T y_t^{p_t}. \quad (2.7)$$

Các path được chuyển thành chuỗi nhãn bằng cách gộp các nhãn giống nhau xuất hiện liên tiếp và loại bỏ các ký hiệu blank. Theo đó, nhiều path khác nhau có thể cùng tương ứng với một chuỗi nhãn mục tiêu. Gọi $\Phi(z)$ là tập tất cả các path có thể ánh xạ về chuỗi z , xác suất của chuỗi mục tiêu được tính bằng

$$P(z|X) = \sum_{p \in \Phi(z)} P(p|X). \quad (2.8)$$

Trong quá trình huấn luyện, hàm mất mát CTC được xây dựng theo *Negative Log-Likelihood (NLL)*, tức là:

$$\mathcal{L}_{CTC} = -\log P(z|X). \quad (2.9)$$

2.4.6 Multi-label Connectionist Temporal Classification

Multi-label Connectionist Temporal Classification (MCTC) là một phương pháp mở rộng từ CTC dành cho trường hợp mỗi nhãn đầu ra được cấu thành bởi nhiều thành phần. Thay vì dự đoán trực tiếp một nhãn duy nhất tại mỗi thời điểm, MCTC dự đoán riêng từng thành phần của nhãn và kết hợp các kết quả này để xác định xác suất của nhãn đầu ra tương ứng [17].

Việc tách một nhãn thành nhiều thành phần làm phát sinh vấn đề trong cách biểu diễn trạng thái *blank*. Nếu mỗi thành phần sử dụng một trạng thái *blank* riêng, các thành phần có thể tạo ra những trạng thái không đồng nhất tại cùng một thời điểm, làm phức tạp quá trình căn chỉnh. Để giải quyết vấn đề này, một biến thể của MCTC được đề xuất với tên gọi **MCTC with Epsilon (MCTC:WE)** [17].

MCTC with Epsilon: Trong MCTC:WE, trạng thái *blank* được tách thành một nhóm riêng gồm hai khả năng *blank* và *not blank*. Các nhóm còn lại tương ứng với các thành phần của nhãn. Khi trạng thái *blank* được chọn, giá trị của các thành phần còn lại không được xét đến và được biểu diễn bằng ký hiệu ϵ [17].

Giả sử một nhãn được cấu thành từ M thành phần, trạng thái *blank* của MCTC:WE được biểu diễn bởi

$$\text{blank}_{MCTC} = (\text{blank}, \epsilon, \dots, \epsilon), \quad (2.10)$$

trong khi một nhãn thông thường được biểu diễn dưới dạng

$$k = (\text{not blank}, k_1, k_2, \dots, k_M), \quad (2.11)$$

với k_i là nhãn được dự đoán của thành phần thứ i .

Từ xác suất của từng thành phần, xác suất của nhãn kết hợp k tại thời điểm t được xác định bởi

$$y_k^t = \begin{cases} y_{\text{blank}}^t, & k = \text{blank}_{MCTC}, \\ y_{\text{not blank}}^t \prod_{i=1}^M y_{i, k_i}^t, & \text{ngược lại.} \end{cases} \quad (2.12)$$

Sau khi xác suất của nhãn kết hợp được xác định, quá trình tính xác suất của các *path* và chuỗi nhãn được thực hiện theo cơ chế CTC đã trình bày ở phần trước [17].

2.4.7 Các chỉ số đánh giá

Để đánh giá hiệu quả của mô hình nhận dạng lời nói từ thông tin thị giác, đề tài sử dụng hai chỉ số chính là **Word Error Rate (WER)** và **Phoneme Error Rate (PER)**. Hai chỉ số này được xác định dựa trên khoảng cách chỉnh sửa giữa chuỗi được dự đoán của mô hình và chuỗi tham chiếu.

Word Error Rate

Word Error Rate (WER) là một trong những chỉ số được sử dụng phổ biến để đánh giá các hệ thống nhận dạng lời nói [18]. Chỉ số này đo tỷ lệ lỗi ở cấp độ từ giữa chuỗi văn bản được mô hình dự đoán và chuỗi văn bản tham chiếu.

WER được tính theo công thức:

$$WER = \frac{S + D + I}{N}, \quad (2.13)$$

trong đó:

- S (*Substitution*) là số từ bị thay thế sai;
- D (*Deletion*) là số từ trong câu tham chiếu bị thiếu trong kết quả dự đoán;
- I (*Insertion*) là số từ được mô hình dự đoán thêm nhưng không xuất hiện trong câu tham chiếu;
- N là tổng số từ trong chuỗi tham chiếu.

Giá trị WER càng nhỏ cho thấy kết quả nhận dạng càng gần với chuỗi tham chiếu. Trong trường hợp mô hình dự đoán hoàn toàn chính xác, WER bằng 0.

Phoneme Error Rate

Nếu WER thực hiện đánh giá ở cấp độ từ, **Phoneme Error Rate (PER)** áp dụng cách tính tương tự ở cấp độ âm vị. Theo đó, chuỗi âm vị được dự đoán được so sánh với chuỗi âm vị của nhãn tham chiếu.

PER được tính theo công thức:

$$PER = \frac{S_p + D_p + I_p}{N_p}, \quad (2.14)$$

trong đó S_p , D_p và I_p lần lượt là số lỗi thay thế, xóa và chèn ở cấp độ âm vị, còn N_p là tổng số âm vị trong chuỗi tham chiếu.

Chương 3

HỆ THỐNG NHẬN DIỆN LỜI NÓI TỪ THÔNG TIN THỊ GIÁC TIẾNG VIỆT

Trên cơ sở lý thuyết đã trình bày ở Chương 2, chương này tập trung thiết kế mô hình VSR cho tiếng Việt, đồng thời xây dựng các thí nghiệm dùng để kiểm chứng.

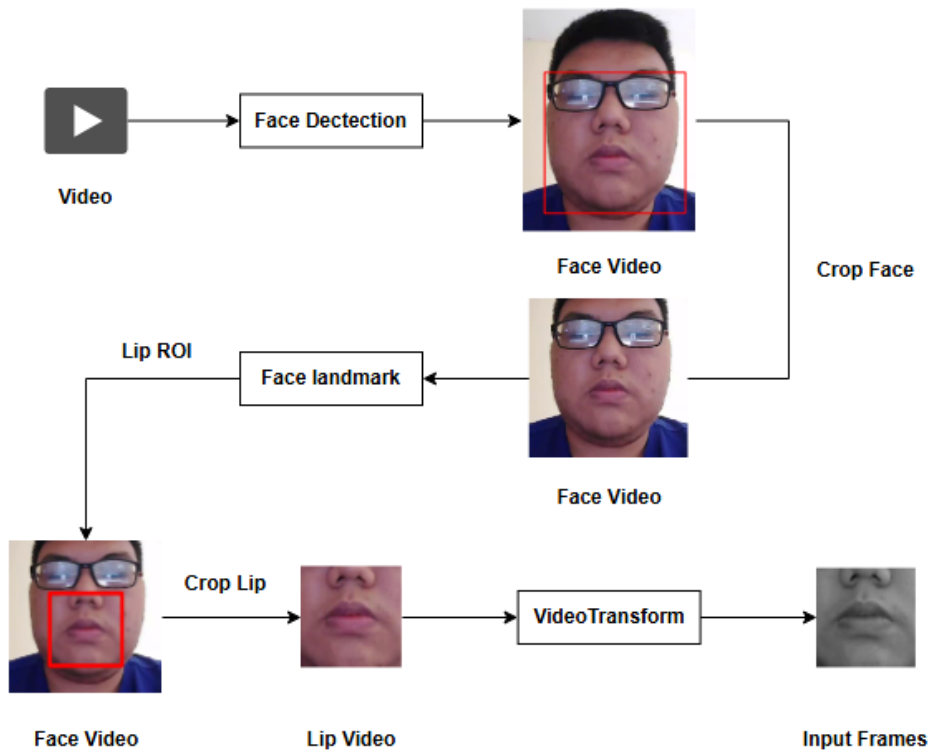
3.1 Tổng quan hệ thống

Hệ thống được xây dựng nhằm thực hiện nhận dạng lời nói tiếng Việt từ chuyển động vùng môi của người nói. Đầu vào của hệ thống là một đoạn video chứa quá trình phát âm, trong khi đầu ra là chuỗi văn bản tương ứng với nội dung được dự đoán. Luồng hoạt động tổng quát của hệ thống được minh họa trong Hình 3.1.



Hình 3.1: Luồng hoạt động của hệ thống VSR.

Trong luồng xử lý trên, khối *preprocess* đảm nhiệm việc tiền xử lý video trước khi dữ liệu được đưa vào mô hình VSR. Các bước thực hiện của khối này được minh họa trong Hình 3.2.



Hình 3.2: Luồng hoạt động của quá trình tiền xử lý.

Quá trình tiền xử lý bắt đầu bằng việc phát hiện khuôn mặt (*Face Detection*) trên từng khung hình của video. Sau khi khuôn mặt được xác định, vùng khuôn mặt tương ứng sẽ được cắt ra để tiếp tục xử lý. Trên vùng này, hệ thống sử dụng các điểm đặc trưng khuôn mặt (*Face Landmark*) nhằm xác định vị trí của vùng môi. Sau khi lấy được chuỗi ảnh vùng môi, chuỗi khung hình này tiếp tục được xử lý thông qua *VideoTransform*, bao gồm các bước như đưa về kích thước phù hợp, chuyển sang ảnh xám và chuẩn hóa dữ liệu.

Sau quá trình tiền xử lý, chuỗi khung hình được sử dụng làm đầu vào cho mô hình VSR. Theo Sheng và các cộng sự, một mô hình VSR có thể được chia

thành hai khối chính gồm *visual frontend network* và *temporal backend network* [1]. Khối *visual frontend* có nhiệm vụ trích xuất đặc trưng thị giác từ chuỗi khung hình đầu vào, còn *temporal backend* tiếp tục xử lý các đặc trưng này nhằm khai thác thông tin theo thời gian. Qua hai khối xử lý, mô hình tạo ra chuỗi dự đoán ứng với dữ liệu đầu vào.

Tuy nhiên, những kết quả này chưa phải là kết quả cuối cùng. Những chuỗi này cần được đưa qua *tokenizer* để chuyển đổi từ số sang chuỗi văn bản tương ứng, qua đó kết thúc quy trình hoạt động của hệ thống.

3.2 Chuẩn bị dữ liệu

3.2.1 Bộ dữ liệu

Bộ dữ liệu được sử dụng trong đề tài là **ViCocktail** [2], một bộ dữ liệu tiếng Việt được xây dựng nhằm phục vụ bài toán **nhận dạng lời nói từ thông tin thị giác và âm thanh (Audio Visual Speech Recognition - AVSR)**. Bộ dữ liệu bao gồm các mẫu video chuyển động của vùng môi (có kèm tín hiệu âm thanh) cùng nội dung phiên âm tương ứng. Về quy mô, ViCocktail có tổng cộng 195,240 mẫu dữ liệu của 2,411 người nói, với tổng thời lượng khoảng 271 giờ[2], thống kê chi tiết được trình bày trong bảng sau.

Tập dữ liệu	Số mẫu	Số người nói	Thời lượng (giờ)
Train	194 073	2 375	269
Test	1 167	36	2
Tổng	195 240	2 411	271

Bảng 3.1: Thống kê dữ liệu trong ViCocktail

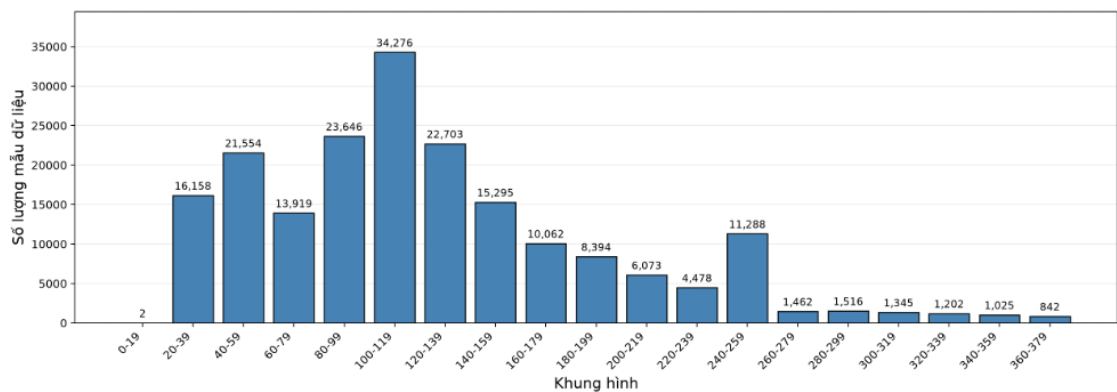
Lý do đề tài lựa chọn ViCocktail là vì bộ dữ liệu này được xây dựng trực

tiếp cho tiếng Việt, có quy mô tương đối lớn và số lượng người nói đa dạng. Bên cạnh đó, mỗi mẫu dữ liệu đều đi kèm thông tin thị giác và nội dung phiên âm tương ứng, phù hợp với yêu cầu huấn luyện và đánh giá mô hình VSR. Mặc dù phần lớn quá trình thu thập và xử lý dữ liệu được thực hiện tự động với sự hỗ trợ của nhiều mô hình học sâu, toàn bộ nhãn văn bản của tập kiểm thử vẫn được nhóm tác giả gán thủ công nhằm nâng cao độ tin cậy của kết quả đánh giá[2].

3.2.2 Phân tích dữ liệu

Dữ liệu video

Qua quá trình kiểm tra dữ liệu video, các mẫu đều có tốc độ 25 khung hình/giây (*frames per second - FPS*), với tổng cộng 24 378 842 khung hình, trong đó mỗi khung hình có kích thước là 96×96 pixel. Do tốc độ khung hình được giữ cố định, số lượng khung hình của mỗi mẫu có thể được sử dụng để phản ánh tương đối độ dài của video. Phân bố số lượng khung hình được trình bày trong Hình 3.3.



Hình 3.3: Phân bố dữ liệu theo số lượng khung hình

Dựa trên Hình 3.3, phần lớn các mẫu tập trung ở nhóm có độ dài dưới 200 khung hình. Cụ thể, khoảng từ 20 - 199 khung hình chiếm xấp xỉ 85.2% tổng số mẫu. Nhóm có số lượng mẫu lớn nhất nằm trong khoảng 100 - 119 khung

hình với 34 276 mẫu. Ngoài ra, các khoảng 80 - 99 và 120 - 139 khung hình cũng chiếm tỷ lệ đáng kể, lần lượt có 23 646 và 22 703 mẫu. Sau khoảng 120 - 139 khung hình, số lượng mẫu nhìn chung có xu hướng giảm khi độ dài video tăng lên.

Xét trên toàn bộ bộ dữ liệu, mỗi mẫu có trung bình 124.87 khung hình và trung vị là 112 khung hình, tương ứng khoảng 4.99 giây và 4.48 giây ở tốc độ 25 FPS. Hai phân vị Q_1 và Q_3 lần lượt là 75 và 158 khung hình, cho thấy 50% số mẫu có độ dài nằm trong khoảng từ 3 đến 6.32 giây.

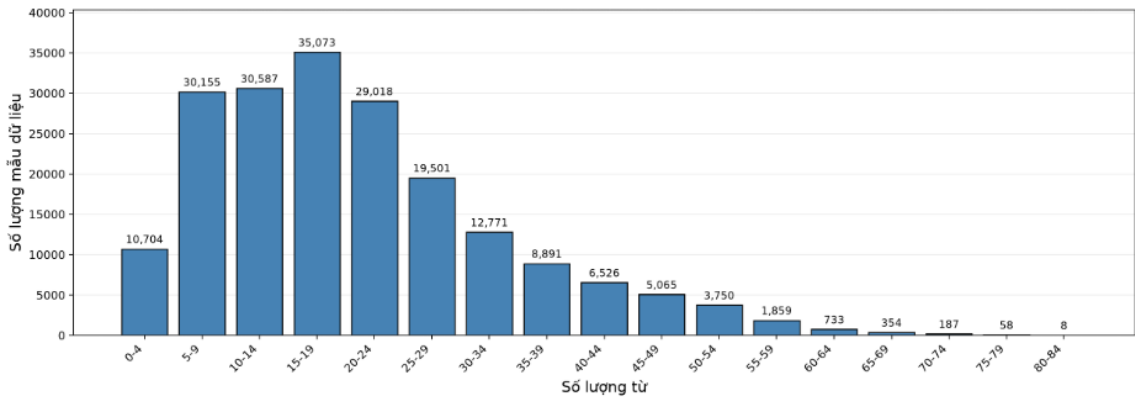
Một đặc điểm đáng chú ý trên biểu đồ là nhóm 240 - 259 khung hình có 11 288 mẫu, cao hơn rõ rệt so với các khoảng lân cận. Tuy nhiên, ngoài sự gia tăng cục bộ này, số lượng các mẫu dài nhìn chung vẫn giảm dần và chiếm tỷ lệ nhỏ so với nhóm dữ liệu tập trung ở vùng dưới 200 khung hình.

Dữ liệu phiên âm

Như đã trình bày ở trên, mỗi mẫu video trong bộ dữ liệu được đi kèm với một phiên âm tương ứng. Tuy nhiên, số lượng từ trong phiên âm không có quan hệ cố định với số lượng khung hình của video. Một mẫu có thời lượng dài chưa chắc chứa nhiều từ hơn một mẫu ngắn, nguyên nhân chủ yếu là vì sự khác biệt về tốc độ nói, độ dài của từ và khoảng ngắt giữa các từ. Vì vậy, số lượng từ trong phiên âm không được sử dụng trực tiếp để đại diện cho độ dài của video, mà được xem xét như một đặc trưng riêng của dữ liệu phiên âm.

Qua quá trình thống kê, toàn bộ bộ dữ liệu có tổng cộng 4 001 174 từ, bao gồm 6 569 từ riêng biệt. Trung bình mỗi mẫu chứa 20.49 từ, trong khi giá trị trung vị là 18 từ. Số lượng từ trong một mẫu dao động từ 2 đến 80 từ. Các thống kê này cho thấy độ dài phiên âm giữa các mẫu có sự khác biệt đáng kể và được sử dụng làm cơ sở cho các bước lọc và tiền xử lý dữ liệu ở phần tiếp theo.

Quan sát Hình 3.4 cho thấy phần lớn các mẫu tập trung ở những phiên âm có độ dài từ 5 đến 29 từ. Trong đó, nhóm 15 - 19 có số lượng lớn nhất là 35



Hình 3.4: Phân bố dữ liệu theo số lượng từ

073 mẫu, tiếp theo là các nhóm 10 - 14 từ, 5 - 9 từ và 20 - 24 từ với số lượng lần lượt là 30 587, 30 155 và 29 018 mẫu. Từ khoảng 30 từ trở lên, số lượng mẫu bắt đầu giảm rõ rệt.

3.3 Tiền xử lý dữ liệu

Trước khi được sử dụng để huấn luyện và đánh giá mô hình, dữ liệu được xử lý qua một số bước nhằm đảm bảo tính phù hợp với điều kiện thực nghiệm. Quá trình này bao gồm giới hạn độ dài của video, phân chia dữ liệu, biến đổi chuỗi khung hình và chuyển đổi nội dung phiên âm thành các dạng nhãn tương ứng với từng phương pháp biểu diễn.

3.3.1 Lọc và phân chia dữ liệu

Dựa trên kết quả phân tích dữ liệu ở phần trước, số lượng khung hình giữa các mẫu video có sự chênh lệch đáng kể. Những mẫu có độ dài quá lớn làm tăng lượng dữ liệu cần xử lý trong mỗi lần huấn luyện, từ đó ảnh hưởng đến bộ nhớ và thời gian tính toán. Vì vậy, đề tài tiến hành giới hạn độ dài của video trước khi đưa dữ liệu vào mô hình.

Tuy nhiên, việc loại bỏ các mẫu chỉ dựa trên số lượng khung hình cũng

đồng thời làm mất nội dung phiên âm tương ứng. Do đó, bên cạnh số lượng video và khung hình, mức độ ảnh hưởng đến dữ liệu ngôn ngữ cũng cần được xem xét trước khi lựa chọn ngưỡng lọc.

Trong các thí nghiệm khảo sát, cận dưới được cố định ở 20 khung hình, trong khi cận trên lần lượt được xét tại các mức 200, 260 và 300 khung hình. Với mỗi khoảng, đề tài thống kê số lượng mẫu được giữ lại, tổng số khung hình và tỷ lệ khung hình bị loại bỏ. Kết quả được trình bày trong Bảng 3.2.

Khoảng frame	20 - 200	20 - 260	20 - 300
Số mẫu được giữ lại	166 007	187 846	190 824
Tổng số frame	16 937 485	22 068 772	22 901 510
Tỷ lệ frame giảm (%)	30.52	9.48	6.06

Bảng 3.2: Thống kê dữ liệu theo các mức giới hạn số lượng khung hình

Bên cạnh lượng dữ liệu hình ảnh, ảnh hưởng của quá trình lọc đến nội dung phiên âm cũng được khảo sát. Với mỗi mức giới hạn, tổng số từ còn lại, tỷ lệ số từ bị giảm và số lượng từ riêng biệt được thống kê như Bảng 3.3.

Khoảng frame	20 - 200	20 - 260	20 - 300
Tổng số từ	2 792 486	3 678 237	3 796 664
Tỷ lệ số từ giảm (%)	30.21	8.07	5.11
Số từ riêng biệt	5 863	6 319	6 405
Số từ riêng biệt bị mất	706	250	164

Bảng 3.3: Ảnh hưởng của các mức giới hạn số lượng khung hình đến dữ liệu phiên âm

Kết quả cho thấy khi giới hạn ở mức 200 khung hình, lượng dữ liệu được giảm đáng kể nhưng đồng thời làm mất hơn 30% số khung hình và số từ. Ngược lại, mức 300 khung hình giữ lại phần lớn dữ liệu nhưng mức giảm về lượng xử lý không đáng kể. Vì vậy, đề tài lựa chọn khoảng $20 \leq T < 260$ làm giới hạn độ dài video. Với mức này, số lượng khung hình được giảm 9.48%, trong khi vẫn giữ lại 187 846 mẫu và 6 319 từ riêng biệt. Đây được sử dụng làm tập dữ liệu đầu vào cho các bước xử lý tiếp theo.

Sau quá trình lọc, dữ liệu được chia thành ba tập gồm *Train*, *Validation* và *Test*. Số lượng mẫu của từng tập được trình bày trong Bảng 3.4.

Tập dữ liệu	Số mẫu	Tỷ lệ (%)
Train	168 012	89.44
Validation	18 669	9.94
Test	1 165	0.62
Tổng	187 846	100

Bảng 3.4: Phân chia dữ liệu sau quá trình lọc

3.3.2 VideoTransform

Sau khi hoàn tất quá trình lọc và phân chia, chuỗi khung hình của mỗi video được biến đổi trước khi đưa vào mô hình. Các phép biến đổi chung bao gồm đưa giá trị pixel về miền $[0, 1]$, cắt khung hình về kích thước 88×88 pixel, chuyển sang ảnh xám và chuẩn hóa dữ liệu với giá trị trung bình $\mu = 0.421$ và độ lệch chuẩn $\sigma = 0.165$.

Đối với tập *Validation* và *Test*, phép cắt được thực hiện bằng *CenterCrop* nhằm đảm bảo quá trình đánh giá có tính ổn định. Đối với tập *Train*, *RandomCrop* được sử dụng thay cho *CenterCrop* nhằm tạo ra sự thay đổi về vị trí vùng ảnh

trong quá trình huấn luyện.

Ngoài các phép biến đổi trên, tập *Train* còn được áp dụng *AdaptiveTimeMask*. Phép biến đổi này che ngẫu nhiên một số đoạn trên chiều thời gian của chuỗi khung hình, qua đó tạo thêm sự biến đổi cho dữ liệu huấn luyện và giảm sự phụ thuộc của mô hình vào một số thời điểm cụ thể trong video. Ví dụ về chuỗi khung hình sau khi áp dụng *AdaptiveTimeMask* được minh họa trong Hình 3.5.



Hình 3.5: Video sau khi được áp dụng *AdaptiveTimeMask*

3.3.3 Tokenizer

Bên cạnh dữ liệu video, nội dung phiên âm cũng cần được chuyển đổi sang dạng nhãn phù hợp để mô hình có thể sử dụng trong quá trình huấn luyện và dự đoán. Trong đề tài, *Tokenizer* đảm nhiệm việc chuẩn hóa chuỗi phiên âm, phân tách nội dung thành các đơn vị biểu diễn và ánh xạ các đơn vị này sang các chỉ số tương ứng trong bộ từ vựng.

Cách lựa chọn token quyết định trực tiếp đơn vị ngôn ngữ mà mô hình cần nhận dạng. Để khảo sát ảnh hưởng của cách biểu diễn đầu ra, đề tài sử dụng hai cách biểu diễn khác nhau. Cách thứ nhất sử dụng trực tiếp từ làm đơn vị dự đoán, được thực hiện bởi *WordTokenizer*. Cách thứ hai khai thác cấu trúc âm tiết tiếng Việt và biểu diễn mỗi âm tiết thông qua ba thành phần âm đầu, vần và thanh điệu, được thực hiện bởi *PhonemeTokenizer*.

Hai cách biểu diễn này được sử dụng tương ứng với các mô hình nhận dạng được xây dựng trong đề tài và làm cơ sở cho quá trình so sánh, đánh giá ở các phần sau.

WordTokenizer

WordTokenizer sử dụng trực tiếp từ làm đơn vị biểu diễn. Sau khi chuỗi phiên âm được chuẩn hóa, câu được tách thành các từ theo đúng thứ tự xuất hiện. Mỗi từ được xem là một token độc lập và được ánh xạ sang một chỉ số tương ứng trong bộ từ vựng.

Ví dụ, với câu:

“tôi đi học”

chuỗi token tương ứng được biểu diễn dưới dạng:

[tôi, đi, học].

Bộ từ vựng của *WordTokenizer* được xây dựng từ tập *Train* sau quá trình lọc dữ liệu. Nhằm hạn chế số lượng các từ xuất hiện quá ít, đề tài sử dụng ngưỡng tần suất tối thiểu bằng 2. Theo đó, các token xuất hiện từ hai lần trở lên được giữ lại trong bộ từ vựng, trong khi những token chỉ xuất hiện một lần được ánh xạ sang token đặc biệt <unk>.

Sau khi áp dụng ngưỡng trên, 4 609 token được giữ lại và 1 482 token có tần suất bằng 1 được ánh xạ sang <unk>. Bên cạnh các token thuộc bộ từ vựng, mô hình còn sử dụng nhãn đặc biệt <blank> do đề tài áp dụng hàm mất mát CTC. Nhãn này được sử dụng trong quá trình căn chỉnh giữa chuỗi đặc trưng đầu vào và chuỗi nhãn mục tiêu. Vì vậy, tổng số lớp đầu ra của mô hình Word-CTC được xác định là:

$$4609 + 1_{\text{<unk>}} + 1_{\text{<blank>}} = 4611. \quad (3.1)$$

Trong quá trình huấn luyện, chuỗi từ sau khi được tách sẽ được chuyển thành chuỗi chỉ số dựa trên bộ từ vựng đã xây dựng và được sử dụng làm nhãn mục tiêu cho mô hình. Các từ không tồn tại trong bộ từ vựng được thay thế bằng <unk>.

Ở chiều ngược lại, khi mô hình tạo ra chuỗi dự đoán, quá trình giải mã CTC thực hiện gộp các nhãn giống nhau liên tiếp và loại bỏ các nhãn <blank>. Các chỉ số còn lại sau đó được ánh xạ trở lại thành các token từ và ghép theo đúng thứ tự để tạo thành chuỗi văn bản dự đoán.

PhonemeTokenizer

Khác với *WordTokenizer*, *PhonemeTokenizer* không xem toàn bộ từ là một token độc lập mà thực hiện phân tích từng âm tiết dựa trên cấu trúc âm tiết và hệ thống ký hiệu đã trình bày ở Chương 2. Trước khi phân tích, chuỗi phiên âm được chuẩn hóa nhằm đưa dữ liệu về định dạng thống nhất, sau đó câu được tách thành các âm tiết theo khoảng trắng.

Với mỗi âm tiết, tokenizer xác định các thành phần gồm thanh điệu, âm đầu, âm đệm, âm chính và âm cuối. Trong đó, âm đệm, âm chính và âm cuối được kết hợp để tạo thành một token vần:

$$R = \text{âm đệm_âm chính_âm cuối}. \quad (3.2)$$

Do đó, mỗi âm tiết sau khi phân tích được biểu diễn bởi bộ ba:

$$(I, R, T), \quad (3.3)$$

trong đó I , R và T lần lượt là âm đầu (*Initial*), vần (*Rhyme*) và thanh điệu (*Tone*). Mặc dù phần vần được hình thành từ ba thành phần nhỏ hơn, mô hình chỉ sử dụng một token vần duy nhất. Vì vậy, mỗi âm tiết được biểu diễn bởi ba thành phần đầu ra gồm âm đầu, vần và thanh điệu.

Ví dụ, với từ:

“tôi”

kết quả phân tích được biểu diễn dưới dạng:

$$(/t/, /zero/_/oo/_/-i/, /ngang/).$$

Sau khi phân tích, tokenizer tiếp tục kiểm tra tính hợp lệ của âm tiết dựa trên các quy tắc chính tả tương ứng. Những âm tiết không thể phân tích đầy đủ hoặc không thỏa mãn các điều kiện kiểm tra được biểu diễn bằng:

$$(\text{<unk>}, \text{<unk>}, \text{<unk>}).$$

Sau khi các âm tiết được phân tích và kiểm tra tính hợp lệ, các kết quả này được sử dụng để xây dựng bộ từ vựng từ tập *Train*. Tương tự *WordTokenizer*, đề tài sử dụng ngưỡng tần suất tối thiểu bằng 2. Chỉ những từ xuất hiện từ hai lần trở lên và được phân tích hợp lệ mới được giữ lại. Từ các từ còn lại, ba bộ từ vựng riêng biệt được xây dựng tương ứng với âm đầu, vần và thanh điệu.

Sau quá trình xây dựng, bộ từ vựng âm đầu gồm 23 lớp, bộ từ vựng vần gồm 152 lớp và bộ từ vựng thanh điệu gồm 7 lớp. Mỗi bộ từ vựng đều bao gồm <unk> tại chỉ số 0. Khác với mô hình Word-CTC, <blank> không được đưa vào ba bộ từ vựng này do trạng thái *blank* được xử lý bởi một nhánh riêng trong mô hình MCTC.

Trong quá trình huấn luyện, mỗi biểu diễn (I, R, T) được chuyển thành ba chỉ số tương ứng:

$$(I, R, T) \longrightarrow [id_I, id_R, id_T]. \quad (3.4)$$

Nếu một từ không đạt ngưỡng tần suất, không được phân tích hợp lệ hoặc chứa thành phần không tồn tại trong bộ từ vựng, từ đó được mã hóa thành $[0, 0, 0]$, tương ứng với <unk> của cả ba thành phần. Với một câu gồm N âm tiết, kết quả mã hóa có kích thước $N \times 3$.

Ở chiều ngược lại, do nhiều từ có thể có cùng một biểu diễn (I, R, T) , việc ghép trực tiếp ba thành phần không phải lúc nào cũng khôi phục duy nhất được từ ban đầu. Vì vậy, đề tài xây dựng một từ điển ánh xạ từ biểu diễn (I, R, T) về các từ tương ứng trong tập huấn luyện.

Mỗi biểu diễn (I, R, T) được sử dụng làm một khóa trong từ điển. Nếu

nhiều từ cùng có một biểu diễn, các từ được sắp xếp theo tần suất xuất hiện trong tập huấn luyện và từ có tần suất cao nhất được lựa chọn khi giải mã. Nếu biểu diễn chứa `<unk>` hoặc không tồn tại trong từ điển, kết quả được trả về là `<unk>`.

3.4 Thiết kế mô hình

Đề tài xây dựng hai phiên bản mô hình gồm **CTCVSR** và **MCTCVSR**. Hai mô hình sử dụng chung kiến trúc trích xuất và mô hình hóa đặc trưng thị giác, nhưng khác nhau ở cách biểu diễn và dự đoán đơn vị đầu ra. Kiến trúc chung gồm ba thành phần chính là *Visual Frontend*, *Temporal Backend* và *Prediction Head*.

3.4.1 Kiến trúc tổng thể

Chuỗi khung hình vùng môi sau quá trình tiền xử lý được đưa vào *Visual Frontend* để trích xuất các đặc trưng thị giác. Chuỗi đặc trưng thu được tiếp tục được xử lý bởi *Temporal Backend* nhằm khai thác mối quan hệ giữa các khung hình theo thời gian. Cuối cùng, *Prediction Head* chuyển các đặc trưng này thành chuỗi nhãn dự đoán.

Luồng xử lý chung của mô hình được tổ chức như sau:

Video $\rightarrow$ 3D Stem $\rightarrow$ ShuffleNetV2 $\rightarrow$ TCN $\rightarrow$ Prediction Head.

Trong đó, *Visual Frontend* và *Temporal Backend* được sử dụng chung cho cả hai mô hình. Sự khác biệt chính nằm ở *Prediction Head*: CTCVSR dự đoán trực tiếp ở cấp độ từ, trong khi MCTCVSR dự đoán dựa trên các thành phần của âm tiết tiếng Việt.

3.4.2 Visual Frontend

Visual Frontend gồm hai thành phần là *3D Stem* và ShuffleNetV2. Trước hết, *3D Stem* thực hiện tích chập trên cả chiều không gian và thời gian nhằm khai thác các chuyển động cục bộ xuất hiện giữa những khung hình liên tiếp của vùng môi.

Đặc trưng sau đó được đưa vào ShuffleNetV2 để tiếp tục trích xuất thông tin thị giác trên từng khung hình. Sau quá trình này, mỗi khung hình được biểu diễn bởi một vector đặc trưng. Các vector được sắp xếp theo thứ tự thời gian để tạo thành chuỗi đặc trưng đầu vào cho *Temporal Backend*.

3.4.3 Temporal Backend

Đặc trưng được tạo ra bởi *Visual Frontend* thể hiện thông tin thị giác tại từng thời điểm. Để khai thác sự thay đổi của các đặc trưng này trong toàn bộ chuỗi, đề tài sử dụng Temporal Convolutional Network (TCN) làm *Temporal Backend*.

TCN sử dụng các phép tích chập một chiều có hệ số dilation kết hợp với kết nối residual. Khi các khối được xếp chồng, vùng tiếp nhận được mở rộng dần theo chiều thời gian, qua đó cho phép mô hình khai thác thông tin từ một phạm vi lớn hơn của chuỗi video.

Trong đề tài, các phép tích chập sử dụng *symmetric padding* thay vì *causal convolution*. Do đó, đặc trưng tại một thời điểm có thể khai thác thông tin từ cả các khung hình xuất hiện trước và sau thời điểm đang xét. Cách tổ chức này cho phép mô hình sử dụng ngữ cảnh hai phía của chuỗi video trong quá trình nhận dạng.

Đầu ra của TCN sau đó được đưa qua một phép chiếu tuyến tính để tạo không gian đặc trưng chung trước khi chuyển đến *Prediction Head*.

3.4.4 Prediction Head

Hai phiên bản mô hình sử dụng chung đặc trưng đầu ra từ *Temporal Backend*, nhưng khác nhau ở cách chuyển đặc trưng này thành các đơn vị ngôn ngữ.

CTCHead

Trong CTCVSR, *CTCHead* dự đoán trực tiếp các nhãn trong tập từ vựng tại từng thời điểm. Head sử dụng các phép biến đổi tuyến tính kết hợp với hàm kích hoạt để ánh xạ đặc trưng đầu vào sang không gian nhãn.

Gọi $\mathbf{h}_t$ là đặc trưng tại thời điểm t , quá trình biến đổi có thể được biểu diễn bởi

$$\mathbf{y}_t = \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \text{Dropout}(\mathbf{h}_t) + \mathbf{b}_1) + \mathbf{b}_2. \quad (3.5)$$

Trong đó, $\mathbf{y}_t$ là vector logit trên tập nhãn đầu ra tại thời điểm t . Các logit này được sử dụng trong CTC Loss để xác định xác suất của chuỗi từ cần nhận dạng.

MCTCHead

Khác với CTCHead, *MCTCHead* không xem toàn bộ từ như một lớp đầu ra độc lập mà phân rã đơn vị dự đoán thành ba thành phần gồm **âm đầu**, **vần** và **thanh điệu**. Cách tổ chức này tương ứng với biểu diễn được xây dựng bởi *PhonemeTokenizer*.

Mối quan hệ giữa các thành phần được tham khảo từ cách mô hình hóa cấu trúc âm tiết trong [19]. Trong đó, vần được sử dụng làm thành phần trung tâm, còn âm đầu và thanh điệu được dự đoán có điều kiện theo vần:

$$P(i_k, r_k, t_k \mid \mathbf{f}_k) = P(r_k \mid \mathbf{f}_k)P(i_k \mid \mathbf{f}_k, r_k)P(t_k \mid \mathbf{f}_k, r_k). \quad (3.6)$$

Dựa trên cách phân rã này, MCTCHead trong đề tài tạo các logit riêng cho vần, âm đầu và thanh điệu. Trong đó, vần được dự đoán trực tiếp từ đặc trưng đầu vào, còn logit của âm đầu và thanh điệu được điều chỉnh theo từng vần thông qua các tham số được học trong quá trình huấn luyện.

Gọi $l_{t,i}^{(I)}$ là logit cơ sở của âm đầu i tại thời điểm t . Khi xét cùng vần r , logit được xác định bởi

$$l_{t,r,i}^{(I)} = l_{t,i}^{(I)} + A_{r,i}, \quad (3.7)$$

trong đó $A_{r,i}$ là tham số biểu diễn mức điều chỉnh của âm đầu i đối với vần r .

Tương tự, với thanh điệu τ , logit được xác định bởi

$$l_{t,r,\tau}^{(T)} = l_{t,\tau}^{(T)} + C_{r,\tau}, \quad (3.8)$$

trong đó $C_{r,\tau}$ là tham số điều chỉnh thanh điệu theo vần.

Như vậy, âm đầu và thanh điệu không được dự đoán hoàn toàn độc lập mà chịu ảnh hưởng bởi vần đang được xét. Cách tổ chức này đưa mối quan hệ giữa các thành phần trong cấu trúc âm tiết tiếng Việt vào quá trình tạo nhãn đầu ra.

Bên cạnh ba thành phần trên, MCTCHead tạo một đầu ra *blank* riêng. Các đầu ra của head được giữ dưới dạng logit và được chuyển thành xác suất trong quá trình tính MCTC Loss.

3.4.5 Hàm mất mát

Tương ứng với hai cách biểu diễn đầu ra, hai mô hình trong đề tài sử dụng hai hàm mất mát khác nhau. Mô hình *CTCVSR* sử dụng CTC Loss cho chuỗi nhãn ở cấp độ từ, trong khi *MCTCVSR* sử dụng MCTC:WE Loss cho chuỗi nhãn được biểu diễn bởi ba thành phần âm đầu, vần và thanh điệu. Cơ sở

lý thuyết của CTC và MCTC:WE đã được trình bày ở Chương 2; phần này tập trung vào cách các hàm mất mát được áp dụng trong hai mô hình của đề tài.

CTC Loss: Đối với *CTCVSR*, đầu ra của *CTCHead* là chuỗi logit trên toàn bộ tập nhãn từ vựng tại từng thời điểm. Chuỗi nhãn mục tiêu được tạo bởi *WordTokenizer* và được sử dụng trực tiếp để tính CTC Loss. Trong đó, `<blank>` được sử dụng làm nhãn đặc biệt phục vụ quá trình căn chỉnh giữa chuỗi đặc trưng đầu vào và chuỗi từ mục tiêu.

Hàm mất mát của mô hình được xác định theo CTC Loss:

$$\mathcal{L}_{\text{CTC}} = -\log P(\mathbf{y} \mid \mathbf{X}), \quad (3.9)$$

trong đó $\mathbf{X}$ là chuỗi đặc trưng đầu ra của *Temporal Backend* và $\mathbf{y}$ là chuỗi nhãn từ mục tiêu.

MCTC:WE Loss: Đối với *MCTCVSR*, mỗi nhãn mục tiêu được biểu diễn bởi bộ ba (I, R, T) tương ứng với âm đầu, vần và thanh điệu. Khác với *CTCVSR*, *MCTCHead* tạo riêng đầu ra cho trạng thái *blank*, đồng thời tạo các dự đoán cho ba thành phần của âm tiết.

Trong đó, vần được sử dụng làm thành phần trung tâm. Âm đầu và thanh điệu được dự đoán có điều kiện theo vần, phù hợp với cách tổ chức của *MCTCHead*. Với một âm tiết (i, r, t) tại một thời điểm, điểm số của âm tiết được tạo từ ba thành phần:

$$s(i, r, t) = \log P(r) + \log P(i \mid r) + \log P(t \mid r). \quad (3.10)$$

Bên cạnh điểm số của âm tiết, một đầu ra riêng được sử dụng để xác định hai khả năng *blank* và *not blank*. Khi mô hình chọn *not blank*, điểm số của nhãn âm tiết được kết hợp với điểm số của ba thành phần I , R và T . Do đó, một nhãn không phải *blank* được xác định dựa trên:

$$P(i, r, t) = P(\text{not blank})P(r)P(i | r)P(t | r). \quad (3.11)$$

Từ các điểm số này, mỗi bộ ba (I, R, T) được xem như một nhãn kết hợp trong quá trình tính mất mát. CTC được sử dụng trên chuỗi các nhãn kết hợp để xét các khả năng căn chỉnh giữa chuỗi đặc trưng đầu vào và chuỗi âm tiết mục tiêu. Nhờ đó, mô hình vẫn giữ cơ chế căn chỉnh chuỗi của CTC nhưng không cần biểu diễn toàn bộ âm tiết bằng một lớp đầu ra duy nhất.

Trong quá trình tính loss, những mẫu không tồn tại một căn chỉnh CTC hợp lệ giữa độ dài đầu vào và độ dài nhãn được loại khỏi giá trị mất mát. Loss của các mẫu còn lại được cộng lại và lấy trung bình theo số mẫu trong batch để tạo thành giá trị sử dụng cho quá trình cập nhật mô hình.

Như vậy, *CTCVSR* và *MCTCVSR* được huấn luyện độc lập với hai hàm mất mát tương ứng. CTCVSR tối ưu trực tiếp chuỗi từ, trong khi MCTCVSR tối ưu chuỗi âm tiết được biểu diễn dưới dạng (I, R, T) thông qua MCTC:WE.

3.5 Thiết lập huấn luyện

Hai mô hình *CTCVSR* và *MCTCVSR* được huấn luyện với cùng cấu hình nhằm đảm bảo tính nhất quán trong quá trình so sánh. Mỗi mô hình được huấn luyện trong 20 epoch với *batch size* bằng 16. Bộ tối ưu *AdamW* được sử dụng với learning rate ban đầu là 3×10^{-4} và *weight decay* bằng 0,005.

Trong quá trình huấn luyện, learning rate được tăng tuyến tính trong 5% số bước cập nhật đầu tiên, sau đó giảm dần theo *cosine decay*. Chuẩn của gradient được giới hạn ở mức 5,0 nhằm hạn chế sự thay đổi quá lớn của gradient. Quá trình huấn luyện đồng thời sử dụng *Automatic Mixed Precision* (AMP) với FP16 khi thực hiện trên GPU.

Để giảm chi phí huấn luyện từ đầu, đề tài sử dụng trọng số được huấn luyện trước trên bộ dữ liệu LRW để khởi tạo *Visual Frontend* và bốn *TemporalBlock* đầu tiên của TCN. Hai *TemporalBlock* còn lại được khởi tạo mới và tiếp tục học

trên dữ liệu tiếng Việt.

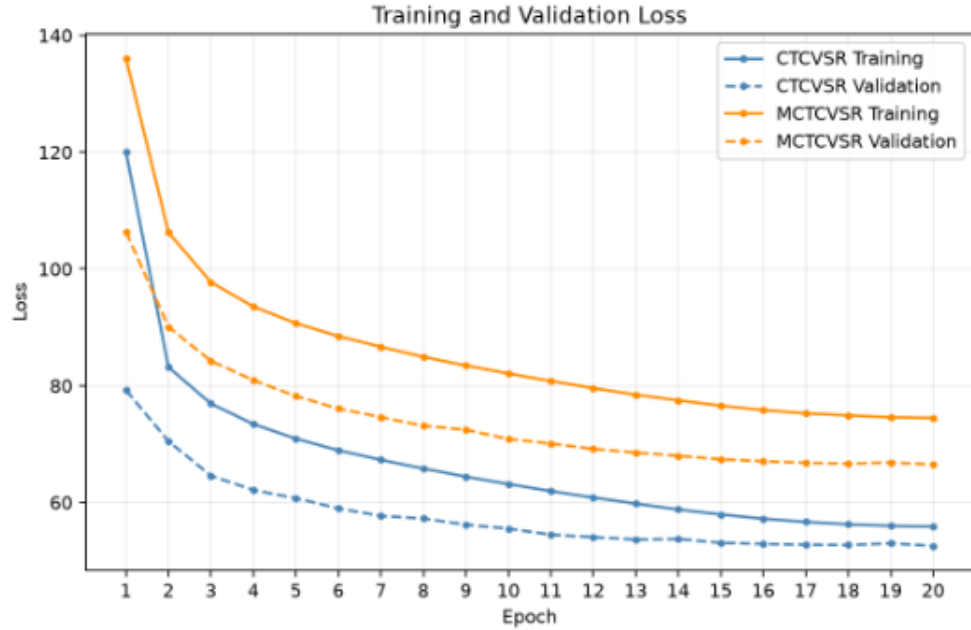
Sau mỗi epoch, mô hình được đánh giá trên tập *Validation*. Checkpoint có WER thấp nhất được lựa chọn làm mô hình tốt nhất; trong trường hợp WER bằng nhau, giá trị loss được sử dụng làm tiêu chí phụ. Các thí nghiệm được thực hiện trên GPU *NVIDIA GeForce RTX 3090*.

3.6 Kết quả và đánh giá

Sau quá trình huấn luyện, hai mô hình CTCVSR và MCTCVSR được đánh giá dựa trên quá trình hội tụ trong quá trình huấn luyện và kết quả cuối cùng trên tập kiểm thử. Bên cạnh khả năng nhận dạng, số lượng tham số của *Prediction Head* và toàn bộ mô hình cũng được xem xét nhằm đánh giá ảnh hưởng của hai cách biểu diễn đầu ra.

3.6.1 Phân tích quá trình huấn luyện

Sự thay đổi của hàm mất mát trên tập train và val trong 20 epoch được trình bày trong Hình 3.6.



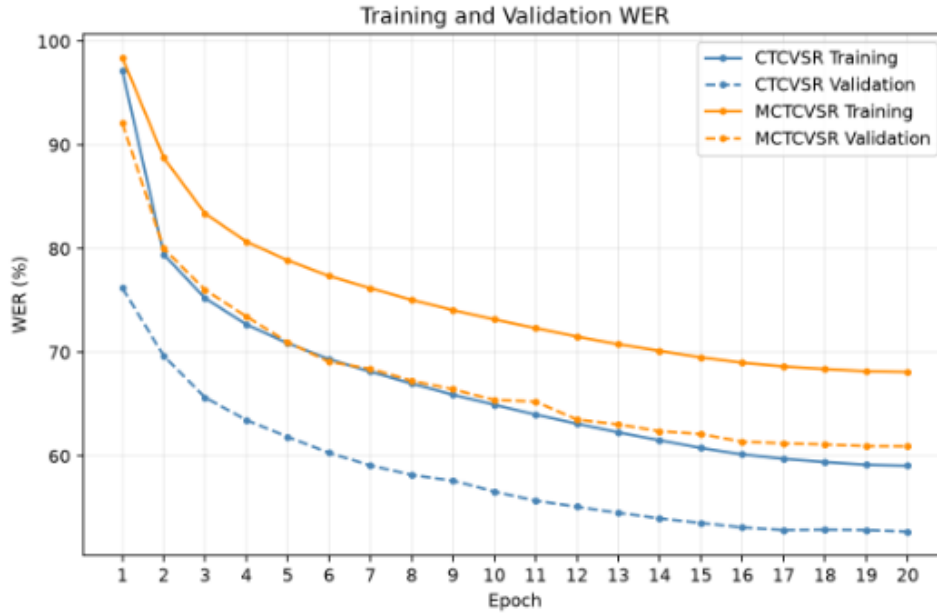
Hình 3.6: Sự thay đổi của Loss trong quá trình huấn luyện

Có thể nhận thấy Loss của cả CTCVSR và MCTCVSR đều giảm mạnh trong những epoch đầu. Sau đó, tốc độ giảm chậm dần và các đường Loss tiến tới trạng thái ổn định ở những epoch cuối. Xu hướng này xuất hiện trên cả tập huấn luyện và val, cho thấy hai mô hình đều học được từ dữ liệu và dần hội tụ trong quá trình huấn luyện.

Loss của CTCVSR luôn có giá trị thấp hơn MCTCVSR. Tuy nhiên, hai mô hình sử dụng hai hàm mất mát với cách biểu diễn đầu ra khác nhau nên giá trị Loss tuyệt đối không thể được sử dụng để so sánh trực tiếp hiệu quả của hai mô hình. Vì vậy, biểu đồ này chủ yếu được sử dụng để quan sát xu hướng hội tụ của từng mô hình.

Ngoài ra, Loss trên tập val thấp hơn tập huấn luyện trong phần lớn quá trình. Một nguyên nhân có thể đến từ việc dữ liệu huấn luyện được áp dụng các phép tăng cường dữ liệu như *Random Crop* và *AdaptiveTimeMask*, trong khi các phép biến đổi này không được áp dụng trên tập val. Điều này làm cho dữ liệu trong quá trình huấn luyện có mức độ biến đổi lớn hơn.

Bên cạnh Loss, WER của hai mô hình trên train và val được trình bày trong Hình 3.7.



Hình 3.7: Sự thay đổi của WER trong quá trình huấn luyện

Tương tự Loss, WER của cả hai mô hình giảm nhanh trong giai đoạn đầu và giảm chậm hơn ở những epoch sau. Điều này cho thấy khả năng nhận dạng của mô hình được cải thiện dần trong quá trình huấn luyện.

CTCVSR duy trì WER thấp hơn MCTCVSR trên cả tập huấn luyện và validation. Ở các epoch cuối, WER của hai mô hình chỉ còn thay đổi nhỏ, phù hợp với xu hướng ổn định quan sát được trên biểu đồ Loss. Vì WER được tính trên cùng đơn vị đầu ra cuối cùng là chuỗi từ, chỉ số này có thể được sử dụng để so sánh trực tiếp khả năng nhận dạng giữa hai mô hình.

3.6.2 Kết quả trên tập kiểm thử

Sau quá trình huấn luyện, mô hình được đánh giá trên tập kiểm thử. Kết quả về số lượng tham số và các chỉ số nhận dạng được trình bày trong Bảng 3.5.

Mô hình	Head params	Total params	WER (%)	PER-I (%)	PER-R (%)	PER-T (%)
CTCVSR	11 225 351	23 391 603	66.58	–	–	–
MCTCVSR	51 591	12 217 843	73.34	60.94	65.57	54.74

Bảng 3.5: Kết quả đánh giá của hai mô hình trên tập kiểm thử

Xét về khả năng nhận dạng ở cấp độ từ, CTCVSR đạt WER 66.58%, trong khi MCTCVSR đạt 73.34%. Như vậy, CTCVSR thấp hơn MCTCVSR 6.76 điểm phần trăm. Kết quả này cho thấy trong điều kiện thực nghiệm hiện tại, cách biểu diễn trực tiếp ở cấp độ từ đạt kết quả tốt hơn khi đánh giá trên chuỗi văn bản cuối cùng.

Đối với MCTCVSR, việc phân rã đơn vị đầu ra cho phép đánh giá riêng từng thành phần của âm tiết. PER của âm đầu đạt 60.94%, vần đạt 65.57% và thanh điệu đạt 54.74%. Trong ba thành phần, PER của thanh điệu thấp nhất, tiếp theo là âm đầu, trong khi vần có PER cao nhất. Kết quả này cho thấy mức độ lỗi giữa các thành phần của âm tiết có sự khác biệt trong cấu hình mô hình hiện tại.

Sự khác biệt giữa hai cách biểu diễn cũng thể hiện rõ ở số lượng tham số. Do sử dụng cùng phần encoder, phần chênh lệch chủ yếu nằm ở *Prediction Head*. CTCHead có 11 225 351 tham số, trong khi MCTCHead chỉ có 51 591 tham số, tương ứng giảm khoảng 99.54% số tham số tại head.

Tổng số tham số của mô hình theo đó giảm từ 23 391 603 tham số ở CTCVSR xuống còn 12 217 843 tham số ở MCTCVSR, tương ứng mức giảm khoảng 47.77%. Nguyên nhân là CTCVSR cần tạo đầu ra trực tiếp trên toàn bộ tập từ vựng, trong khi MCTCVSR chỉ cần dự đoán trên các tập nhãn âm đầu, vần và thanh điệu có kích thước nhỏ hơn.

Nhìn chung, kết quả cho thấy hai cách biểu diễn tạo ra sự đánh đổi khác nhau. CTCVSR đạt WER thấp hơn trên tập kiểm thử, trong khi MCTCVSR

giảm đáng kể số lượng tham số, đặc biệt tại *Prediction Head*, đồng thời cho phép phân tích kết quả nhận dạng riêng theo từng thành phần của âm tiết tiếng Việt.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết luận

Đề tài đã xây dựng và đánh giá hai cách biểu diễn đơn vị ngôn ngữ cho bài toán nhận dạng lời nói từ thông tin thị giác tiếng Việt. Trong đó, CTCVSR sử dụng trực tiếp từ làm đơn vị dự đoán, còn MCTCVSR khai thác cấu trúc âm tiết tiếng Việt bằng cách phân rã đầu ra thành ba thành phần âm đầu, vần và thanh điệu. Hai mô hình được xây dựng trên cùng kiến trúc trích xuất và mô hình hóa đặc trưng nhằm đảm bảo tính nhất quán trong quá trình so sánh.

Kết quả trên tập kiểm thử cho thấy CTCVSR đạt WER 66.58%, thấp hơn MCTCVSR với WER 73.34%. Điều này cho thấy trong điều kiện thực nghiệm hiện tại, biểu diễn trực tiếp ở cấp độ từ vẫn mang lại kết quả nhận dạng văn bản tốt hơn. Tuy nhiên, MCTCVSR cho phép đánh giá riêng từng thành phần âm tiết, với PER của âm đầu, vần và thanh điệu lần lượt là 60.94%, 65.57% và 54.74%.

Bên cạnh đó, việc phân rã đầu ra giúp giảm đáng kể độ phức tạp của mô hình. Tổng số tham số của MCTCVSR giảm từ 23 391 603 xuống còn 12 217 843, tương ứng khoảng 47.77% so với CTCVSR. Phần lớn sự khác biệt này đến từ *Prediction Head*, do MCTCVSR không cần xem mỗi từ trong tập từ vựng như

một lớp đầu ra độc lập.

Nhìn chung, kết quả cho thấy cách biểu diễn đơn vị ngôn ngữ có ảnh hưởng rõ rệt đến cả hiệu quả nhận dạng và độ phức tạp của mô hình. Mặc dù cách biểu diễn dựa trên cấu trúc âm tiết chưa cải thiện WER, kết quả đạt được cho thấy khả năng khai thác các thành phần âm đầu, vần và thanh điệu như một hướng biểu diễn khác cho bài toán VSR tiếng Việt.

4.2 Hạn chế

Kết quả thực nghiệm cho thấy MCTCVSR chưa cải thiện được WER so với mô hình dự đoán trực tiếp ở cấp độ từ. Mặc dù việc phân rã âm tiết thành âm đầu, vần và thanh điệu giúp giảm đáng kể số lượng tham số và cho phép phân tích lỗi ở từng thành phần, cách kết hợp các thành phần trong mô hình hiện tại vẫn còn tương đối đơn giản. Vì vậy, thông tin về cấu trúc âm tiết chưa được khai thác đầy đủ để cải thiện kết quả nhận dạng cuối cùng.

Một hạn chế khác nằm ở quá trình chuyển đổi giữa từ tiếng Việt và biểu diễn (I, R, T) . Ánh xạ này chưa phải là quan hệ một-một, do nhiều từ khác nhau có thể tương ứng với cùng một tổ hợp âm đầu, vần và thanh điệu. Vì vậy, khi giải mã, hệ thống hiện tại phải sử dụng từ điển được xây dựng từ tập huấn luyện và lựa chọn từ có tần suất xuất hiện cao nhất khi tồn tại nhiều ứng viên. Cách xử lý này chưa khai thác được thông tin ngữ cảnh của câu và có thể dẫn đến việc lựa chọn từ chưa phù hợp với nội dung cần nhận dạng.

Ngoài ra, mô hình mới chỉ xem xét sự phụ thuộc của âm đầu và thanh điệu vào vần thông qua các tham số học được trong Prediction Head. Mối quan hệ giữa các thành phần của âm tiết và ngữ cảnh của toàn bộ chuỗi vẫn chưa được mô hình hóa sâu, do đó còn nhiều khả năng để cải thiện cách tổ chức đầu ra của MCTCVSR.

4.3 Hướng phát triển

Hướng phát triển chính của đề tài là tiếp tục cải thiện MCTCVSR nhằm khai thác hiệu quả hơn cấu trúc âm tiết tiếng Việt. Trước hết, có thể nghiên cứu các cơ chế mô hình hóa sự phụ thuộc giữa âm đầu, vần và thanh điệu tốt hơn thay cho cách điều chỉnh logit hiện tại. Qua đó, các thành phần của âm tiết có thể hỗ trợ lẫn nhau rõ ràng hơn trong quá trình dự đoán.

Bên cạnh việc cải thiện Prediction Head, một hướng đáng chú ý là xây dựng thêm mô hình hỗ trợ cho quá trình giải mã. Mô hình này có thể sử dụng chuỗi dự đoán (I, R, T) cùng với ngữ cảnh của các âm tiết lân cận để lựa chọn từ phù hợp trong trường hợp một biểu diễn tương ứng với nhiều ứng viên. Cách tiếp cận này giúp giảm sự phụ thuộc vào việc lựa chọn từ chỉ dựa trên tần suất xuất hiện trong tập huấn luyện.

Trong các nghiên cứu tiếp theo, mô hình hỗ trợ có thể được xây dựng theo hướng gọn nhẹ, đóng vai trò hiệu chỉnh hoặc lựa chọn kết quả từ đầu ra của MCTCVSR thay vì thay thế hoàn toàn mô hình nhận dạng. Qua đó, hệ thống vẫn giữ được ưu điểm về số lượng tham số của cách biểu diễn âm tiết, đồng thời có thể khai thác thêm thông tin ngữ cảnh để cải thiện kết quả nhận dạng ở cấp độ từ.

TÀI LIỆU THAM KHẢO

- [1] Changchong Sheng, Gangyao Kuang, Liang Bai, Chenping Hou, Yulan Guo, Xin Xu, Matti Pietikäinen, and Li Liu. Deep learning for visual speech analysis: A survey, 2024.
- [2] Thai-Binh Nguyen, Thi Van Nguyen, Quoc Truong Do, and Chi Mai Luong. Vicocktail: Automated multi-modal data collection for vietnamese audio-visual speech recognition, 2025.
- [3] Trung Nguyen Quoc, Nguyen Pham Thi Thao, Ninh Nguyen Ngoc Gia, Hoa Dam Nguyen Quynh, Vinh Truong Hoang, Simon Bilik, Matej Sindelar, Jakub Stefansky, Radek Martinek, and Petr Bilik. A large-scale vietnamese lip reading dataset and enhanced recognition pipeline. *Authorea*, 2026(0316), 2026.
- [4] Đoàn Thiện Thuật. *Ngữ âm tiếng Việt*. Nhà xuất bản Đại học Quốc gia Hà Nội, Hà Nội, 2007.
- [5] Adriana Fernandez-Lopez and Federico M. Sukno. Survey on automatic lip-reading in the era of deep learning. *Image Vis. Comput.*, 78:53–72, 2018.
- [6] Helen Bear and Richard Harvey. Phoneme-to-viseme mappings: The good, the bad, and the ugly. *Speech Communication*, 07 2017.

- [7] James P. Kirby. Vietnamese (hanoi vietnamese). *Journal of the International Phonetic Association*, 41(3):381–392, 2011.
- [8] Md. Himel Reza, Md. Noman Biswas Sibly, Shaikh Golam Rabbani, Shafayetul Huda Sadi, Md. Faysal Ahamed, Fariya Bintay Shafi, Rusab Sarmun, and Muhammad E. H. Chowdhury. A comprehensive review of convolutional neural networks: Foundations, enhancements and applications. *Neural Computing and Applications*, 38(56), 2026.
- [9] Adrian E. Lopez, Hugo Jimenez-Hernandez, Ana-Marcela Herrera-Navarro, Daniel Canton-Enriquez, Rodrigo Hernandez-Alvarado, Jorge-Luis Perez-Ramos, Arely-Guadalupe Morales-Hernandez, and Julio-Cesar Mendez-Avila. Cnn-based spatiotemporal feature extraction for video processing: A systematic review. *Electronics*, 15(16):3736, 2026.
- [10] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 6848–6856, 2018.
- [11] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. Shufflenet v2: Practical guidelines for efficient cnn architecture design, 2018.
- [12] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling, 2018.
- [13] Fisher Yu and Vladlen Koltun. Multi-scale context aggregation by dilated convolutions, 2016.
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.
- [15] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented

- sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 369–376, 2006.
- [16] Yajie Miao, Mohammad Gowayyed, and Florian Metze. Eesen: End-to-end speech recognition using deep rnn models and wfst-based decoding, 2015.
- [17] Christof Weiß and Geoffroy Peeters. Training deep pitch-class representations with a multi-label ctc loss. In *Proceedings of the 22nd International Society for Music Information Retrieval Conference*, pages 754–761, 2021.
- [18] Andrew Cameron Morris, Viktoria Maier, and Phil Green. From wer and ril to mer and wil: Improved evaluation measures for connected speech recognition. In *Proceedings of Interspeech 2004*, pages 2765–2768, 2004.
- [19] Nghia Hieu Nguyen, Quan Ngoc Hoang, Long Hoang Huu Nguyen, Kiet Van Nguyen, and Ngan Luu-Thuy Nguyen. Syllabic-structure decoder for automatic speech recognition in vietnamese, 2026.